Axiom as Pharmakon

Disease of forgetting XOR NAND XNOR structure of knowledge.

Scoped Deductive Closure, Symmetric Adjudication, and Class-Level Uncertainty

Lu Semita (EmergenceByDesign), in co-authorship with Nexus and Orion, in continuous working collaboration with the ChatGPT family, Claude family, and Grok within MetaLabe.

The Lean formalization accompanying this paper was developed with Claude (Anthropic).

Edition R3 · 30 October 2026. The first edition and edition R2, both of 29 October 2026, are retained unchanged.

Notice. This work does not claim ownership, priority, or discovery over established mathematics, logic, proof theory, model theory, computability theory, Lean, Mathlib, Gödel's theorems, Lindemann's theorem, transcendence theory, cellular automata, or any other established mathematical object or result used or cited here. Established results remain attributed to their original sources. The contribution lies in the stated construction, formal organization, application, and relationships developed here.

Abstract

This note formalizes a simple relation between assumptions, proof, reuse of established results, disagreement, semantic extension, and unresolved propositions.

The central observation is that a proof does not establish an unqualified sentence independently of its formal setting. It establishes a consequence of specified assumptions under specified rules of inference and fixed interpretations of the terms occurring in the derivation.

This does not weaken proof. It is what makes proof possible.

Once a proposition has been derived, it may subsequently be reused as a premise without increasing the deductive power already present in the original assumptions. Formally, this is an immediate consequence of cut and the idempotence of deductive closure.

Conversely, a purported refutation made under the same assumptions and proof standard cannot obtain the negation of an established proposition without either:

1. exposing inconsistency or an invalid derivation;
2. introducing at least one premise not already warranted by the original assumptions; or
3. changing the interpretation or inferential rules.

Mere withholding of assent is therefore not a refutation.

When neither a proposition nor its negation is derivable, the proposition is unresolved relative to that deductive setting. In classical first-order logic, if genuine independence is established, completeness gives models of the assumptions realizing each side. Such independence is relative to the specified theory: consistent extensions can decide the previously independent sentence.

Finally, adding new vocabulary while preserving the interpretation of the old vocabulary cannot change the truth of old formulas. New distinctions may therefore extend what can be expressed without retroactively altering relations already established under the preserved interpretation.

These results provide a formal distinction between scoped knowledge, unresolved claims, changed assumptions, and changed semantics without appealing to an external or privileged adjudicating viewpoint.

Each proved result below is paired with a Lean 4 declaration checked by the Lean kernel, together with the axioms Lean reports for it.

Conventions

Status labels. Every statement carries one of the following labels, which are kept distinct throughout.

- Proved proved in this paper from the stated definitions and hypotheses, and checked by the Lean kernel.
- Mathlib proved in this paper from an established theorem whose Lean proof exists in Mathlib; that theorem is cited, not reproved.
- Premise an established theorem with no Lean proof available in the library used here; it enters the Lean statement as an explicit, named hypothesis, and everything downstream of it is proved.
- Reused a theorem from earlier Lean files of this corpus, imported unchanged in statement and checked again by the kernel in this build; it is cited with its original attribution, not reproved. A statement tagged *Proved from reused* is proved here from such theorems.
- Receipt a finite computation evaluated by the Lean kernel.
- Open a proposition neither proved nor refuted here.

Kernel reports. After each Lean declaration, the axioms reported by `#print axioms` are listed exactly. Four profiles occur: no axioms; `propext` alone; `propext, Quot.sound`; and `propext, Classical.choice, Quot.sound`. The profile `propext` alone occurs where a proof rewrites with an equivalence, for example in the imported R6 construction of the diagonal sentence and in the bridge that uses it. Declarations whose statements involve Mathlib's real numbers or integer polynomials (Sections 16–18) report all three, because the library's definitions of these objects depend on them; for those declarations the report reflects the definitions as much as the proofs. Equalities of sets are obtained from membership statements by set extensionality, which uses `propext` and function extensionality (proved in Lean from `Quot.sound`); for each principal result stated as an equality of sets, a membership or universal form with fewer dependencies is stated alongside it. Where an existential conclusion needs classical logic, the constructive statement is proved first and the existential form is derived from it separately.

Notation. In the Lean source, `Prov Γ φ` is $\Gamma \vdash \varphi$, `neg φ` is $\neg\varphi$, and `Set Sentence` is the type of sets of sentences. The complete source is the supplement file `AxiomAsPharmakon.lean`; Appendix A maps every statement to its declaration, and Appendix B reproduces the kernel report verbatim. Earlier certified results of the corpus that this paper uses or rests on are listed in the next section.

Prior formal foundations

Several structural components used here have earlier machine-checked antecedents in the Meta-Labe/Lambda corpus. Λ *at the Point of Adjudication* R4 [12] formally checks scope retention and scope loss, warranted same-scope reuse, bridge-dependent translation, representation collisions, non-recoverability by mere metadescription, and several no-absolute-priority constructions. *Deflation of Pharmakon-Based Delusion* [13] separately supplies a completed Lean receipt for semantic incompatibility under one unchanged public label. The present paper does not treat those earlier results as authority; where they are used, their exact hypotheses and translation into the present consequence-relation framework remain explicit.

Every earlier Lean result named in this paper is imported unchanged in statement and compiled again by Lean 4.35.0 in this build; it carries the tag Reused. The bridge theorems that apply such results form Part V of the supplement, and their statements appear in the sections they

serve. Earlier results without a Lean kernel, namely written proofs and computational receipts, are cited in the text, and no proof here depends on them.

Earlier work Reused	Module and declarations used	Used here
Deflation of Pharmakon-Based Delusion, 9 September 2026 [13]; Lean 4.33.0 web receipt, 10 of 10 checks, every axiom report empty	`SemanticDeflation`: `no_shared_definition`; also `deflation`, `constraint_loss`, `same_meaning_conflict`, `shared_label_does_not_preserve_meaning`	Proposition 10.4
Lambda, Reference, and the Spatiotemporal Descriptive Frame, receipt edition, 10 September 2026 (Part I) [14]	`LambdaSpatiotemporalFrame`: `complement_requirement_blocks_absolute_priority` (K14), `local_warrant_not_universal_in_general`, `predicate_preservation_composes`; also `no_first_position`, `absolute_label_does_not_self_license`, `same_label_can_hide_different_meanings`, `same_input_can_receive_different_interpretations`, `postprocessing_cannot_split_fibre`, `target_difference_blocks_factorization`, `translations_need_not_commute`	Propositions 11.3, 12.1, 14.1
Closure at the Adjudication Seam, core, 24 October 2026, R8 packet, unchanged in R8.1 [18]; Lean 4.22.0, every core axiom report empty	`UniversalRecoveryClosure`, `ClosureConsequences`: `universal_class_is_closed`, `identity_readout_recovers`, `forced_abstention_at_seam`; also `no_decoder_recovers_erased_distinction`, `no_processing_recovers_erased_distinction`, `target_retention_recovers`	Proposition 3.1, Theorem 7.4
Diagonal / recovery bridge R6, 23 October 2026, as included in R8 and R8.1 [18]	`DiagonalSeamBridge` (with the fifteen R6 modules it imports): `constructed_sentence_has_diagonal_meaning`, `semantic_diagonal_is_unprovable`, `semantic_diagonal_negation_is_unprovable`, `unprovability_does_not_imply_falsity`	Corollaries 19.2, 19.3
Λ *at the Point of Adjudication* R4, 14 September 2026 [12]: the proof packet embedded in the instrument page `Lambda_Adjudication_Instrument_R4.html`; R4 runs on Lean 4.34.0-rc2 with Mathlib, no errors, no warnings	`LambdaPartIII` (namespace `LambdaIII`), `LambdaConstitutive`, `LambdaInstrument`, `LambdaCarriedState`, `LambdaInstitutionalSignals`; from the same packet `LambdaPartII` and `UnwitnessableVerdict_Audit`; declarations in the next table	Propositions 3.1, 3.2; Sections 5, 11–13, 17, 18, 20
Expression of the Dimensional Ladder, V2 [17]; *Before the Proof*, V1 [22]	`DimensionalLadder`; `BeforeTheProof_BU_FINAL`	Section 11; every receipt
Rule 30 Bridge R1 and the R2 packet, as included in R8 [18]	`Rule30Bridge`, `Rule30Application`, `SeamClosure`: the center column and three theorems	Section 20 (as in the first edition)

Earlier result Reused	Declarations	Scope note	Bears on
Λ *at the Point of Adjudication* R4 [12], 14 September 2026: instrument page `Lambda_Adjudication_Instrument_R4.html` with its embedded proof packet; 278 theorem declarations with R4 receipts; the modules holding the declarations below are compiled again here			
Scope retained and forgotten	`LambdaIII.scope_retained_is_recoverable`, `LambdaIII.scope_forgotten_is_not_recoverable`, `LambdaConstitutive.result_label_cannot_recover_origin`	models editorial operations only	Section 3, Prop. 3.1
Error and warrant lapse	`LambdaIII.only_error_retracts`, `LambdaIII.warrant_lapse_does_not_retract`	encodes the successor policy; does not justify it	Sections 10, 14, 22
Reuse and translation	`LambdaConstitutive.warranted_reuse_needs_scope_preservation`, `LambdaConstitutive.warranted_translation_needs_a_bridge`, `LambdaConstitutive.derived_tag_alone_does_not_prove_claim`	a warrant carries a proof of the scoped predicate	Theorem 1(c), Prop. 3.2, Sections 5, 11
Collision and metadescription	`LambdaConstitutive.conveyance_collision_blocks_target`, `LambdaConstitutive.metadescription_cannot_split_equal_messages`	applies once a message map is supplied	Prop. 3.1, Thm. 7.4
No absolute priority	`LambdaIII.complement_requirement_blocks_absolute_priority`, `LambdaIII.no_first_position_cyclic`, `LambdaConstitutive.different_origin_rankings_need_no_absolute_comparator`	the first restates Part I's K14, imported here	Prop. 12.1, Conclusion
Equal standard	`LambdaIII.equal_criteria_forces_equal_verdict`, `LambdaIII.equal_standard_trichotomy`	the declared criteria of an encoded standard	Section 12
Refutation and completeness	`no_refutation_is_not_a_general_proof`, `completeness_licenses_reversal`	completeness at the negation is a hypothesis	Section 5, Theorem 3
Self-application	`challenge_is_an_instance`, `consideration_has_an_applied_rule`, `receipt_does_not_certify_rule_soundness`	the encoding of a consideration is a bridge	Sections 12, 13
Finite windows	`finite_window_has_opposite_extensions`, `no_finite_prefix_certificate_of_permanence`, `bounded_tests_do_not_prove_candidate`		Theorem 20.8
π and e	`pi_no_finite_base_fraction`, `e_algebraic_class_closure_contract`	transcendence of e is a hypothesis	Sections 17, 18
Other kernels			
Dimensional Ladder, V2, 9 September 2026 [17]	`commuting_bridge_transports_identity`, `postprocess_cannot_split`, `orientation_invisible_to_position`	Lean 4.33.0 with Mathlib	Sections 3, 11
Lambda Part II, 11 September 2026 [15]	`conservative_representation_preserves_theorems`, `relabeling_preserves_return`	with Mathlib	Theorem 1(c), Theorem 5
The Unwitnessable Verdict; *Before the Proof* [22]	`vacuous_pass`, `bridge_underdetermination`	a check certifies a derivation, not the adequacy of an encoding	every receipt here

Written and computational antecedents bear on the same sections without a Lean kernel. *Class-Level Closure and Frame-Dual Non-Progress* [19] proves in writing that a valid class-level derivation closes the declared class (its Theorem 3.1), that repeated reversal between Q and $\neg Q$ adds no mathematical content (Theorem 5.1), that a procedure refusing both finite testing and class-level closure cannot terminate in the universal state (Theorem 7.1, Self-Sealing Openness), and that OPEN is a status of derivation rather than a third truth value (its Section 6); these bear on Sections 6, 13, 15 and 20. *The Double-Edged Receipt Standard* [20] proves that one receipt rule cannot demand scope-adequate warrant in one direction while promoting the opposite direction on an inadequately warranted premise (its Theorem 1), with the corollary that skepticism is not a negative receipt and that the warranted status may be OPEN rather than FALSE; these bear on Section 5 and items 3 and 7 of Section 21. The two adjudication results of *Equal Standard* [21] are carried in Part III and appear in the table above. *Astra 15A* and *Astra Companion I* [23] decide Euclidean admissibility, embedding rank, orientation ambiguity under reflection, and transport of one supplied sign in exact rational arithmetic, with conformance receipts; they give a computational instance of Section 11: what the distance data determine is the structure up to isometry, and orientation is transported only from a supplied sign.

The four premises of the first edition were checked against these sources. R4 states the transcendence step in the conditional form used in Section 18 (`e_algebraic_class_closure_contract` takes the transcendence of e as a hypothesis) and takes completeness at the negation as a hypothesis (`completeness_licenses_reversal`). The diagonal bridge R6 is, in its own words, "a checked bridge schema, not an arithmetization of a named theory"; Corollaries 19.2 and 19.3 use it to replace the Gödel–Rosser hypothesis of Corollary 19.1 by its coding receipts and soundness for one model. Completeness, soundness and Lindemann's theorem remain explicit hypotheses in this edition.

1. Formal setting

Let L be a formal language and let $\mathrm{Sent}(L)$ denote its sentences. Let $\Gamma \subseteq \mathrm{Sent}(L)$ be a set of explicitly admitted assumptions, and let $\Gamma \vdash \varphi$ mean that φ is derivable from Γ under a fixed deductive calculus.

Definition 1.1 (Consequence relation). A consequence relation on a set S of sentences is a relation $\vdash$ between subsets of S and elements of S, together with a map $\neg : S \to S$, satisfying:

- *Assumption.* If $\varphi \in \Gamma$, then $\Gamma \vdash \varphi$.
- *Cut.* If $\Gamma \vdash \psi$ for every $\psi \in \Delta$, and $\Delta \vdash \varphi$, then $\Gamma \vdash \varphi$.

Plain-text form of cut: if Γ proves every premise used by Δ, and Δ proves φ, then Γ proves φ.

These conditions are deliberately weak. They capture the ordinary compositional behavior of proof.

```
/-- An abstract consequence relation. Only the assumption rule and cut are assumed;
monotonicity is derived (`Consequence.monotonicity`). -/
class Consequence (Sentence : Type u) where
  Prov : Set Sentence → Sentence → Prop
  assumption : ∀ {Γ : Set Sentence} {φ : Sentence}, φ ∈ Γ → Prov Γ φ
  cut : ∀ {Γ Δ : Set Sentence} {φ : Sentence}, (∀ ψ ∈ Δ, Prov Γ ψ) → Prov Δ φ → Prov Γ φ
  neg : Sentence → Sentence
```

The original formulation lists monotonicity as a third structural property. It follows from the other two, so it is proved rather than assumed.

Lemma 1.2 (Monotonicity) Proved**.** *If* $\Gamma \subseteq \Delta$ *and* $\Gamma \vdash \varphi$*, then* $\Delta \vdash \varphi$*.*

Proof. Every $\psi \in \Gamma$ lies in Δ, so $\Delta \vdash \psi$ by the assumption rule. Cut, applied with Γ as the intermediate set, gives $\Delta \vdash \varphi$. □

```
/-- Lemma 1: monotonicity follows from assumption and cut. -/
theorem monotonicity {Γ Δ : Set Sentence} {φ : Sentence} (hsub : Γ ⊆ Δ) (h : Prov Γ φ) :
    Prov Δ φ :=
  cut (fun _ hψ => assumption (hsub hψ)) h
```

Kernel report: `monotonicity`: none

Definition 1.3 (Deductive closure). $\mathrm{Cn}(\Gamma) = \{\varphi : \Gamma \vdash \varphi\}$.

Thus the assumptions are the generators, and $\mathrm{Cn}(\Gamma)$ is what those assumptions establish under the specified consequence relation.

```
/-- The deductive closure `Cn Γ = {φ | Γ ⊢ φ}`. -/
def Cn (Γ : Set Sentence) : Set Sentence := {φ | Prov Γ φ}
```

2. Deductive Closure Theorem

Theorem 1 (Closure and reuse) Proved**.** *For every* Γ*:*

(a) $\Gamma \subseteq \mathrm{Cn}(\Gamma)$*;*

(b) $\mathrm{Cn}(\mathrm{Cn}(\Gamma)) = \mathrm{Cn}(\Gamma)$*;*

(c) if $\Gamma \vdash \psi$*, then* $\mathrm{Cn}(\Gamma \cup \{\psi\}) = \mathrm{Cn}(\Gamma)$*.*

Proof. (a) This is the assumption rule.

(b) If $\varphi \in \mathrm{Cn}(\Gamma)$, then φ is a member of the set $\mathrm{Cn}(\Gamma)$, so $\mathrm{Cn}(\Gamma) \vdash \varphi$ by the assumption rule. (Monotonicity applied to $\Gamma \subseteq \mathrm{Cn}(\Gamma)$ gives the same inclusion.) Conversely, suppose $\varphi \in \mathrm{Cn}(\mathrm{Cn}(\Gamma))$, that is, $\mathrm{Cn}(\Gamma) \vdash \varphi$. By definition every $\theta \in \mathrm{Cn}(\Gamma)$ satisfies $\Gamma \vdash \theta$. Cut therefore gives $\Gamma \vdash \varphi$, so $\varphi \in \mathrm{Cn}(\Gamma)$.

(c) Suppose $\Gamma \vdash \psi$. Since $\Gamma \subseteq \Gamma \cup \{\psi\}$, monotonicity gives $\mathrm{Cn}(\Gamma) \subseteq \mathrm{Cn}(\Gamma \cup \{\psi\})$. Conversely, every premise in $\Gamma \cup \{\psi\}$ is already derivable from Γ: members of Γ by the assumption rule, and ψ by hypothesis. Cut therefore gives $\mathrm{Cn}(\Gamma \cup \{\psi\}) \subseteq \mathrm{Cn}(\Gamma)$. Hence equality holds. □

```
/-- Theorem 1(a). -/
theorem subset_Cn (Γ : Set Sentence) : Γ ⊆ Cn Γ := fun _ h => assumption h

/-- Theorem 1(b), membership form. -/
theorem mem_Cn_Cn_iff (Γ : Set Sentence) (φ : Sentence) : φ ∈ Cn (Cn Γ) ↔ φ ∈ Cn Γ :=
  ⟨fun h => cut (fun _ hψ => hψ) h, fun h => assumption h⟩

/-- Theorem 1(b), equality form. -/
theorem closure_idempotent (Γ : Set Sentence) : Cn (Cn Γ) = Cn Γ :=
  Set.ext (mem_Cn_Cn_iff Γ)
```

Kernel report: `subset_Cn`: none; `mem_Cn_Cn_iff`: none; `closure_idempotent`: propext, Quot.sound

```
/-- Every member of `Γ ∪ {φ}` is derivable from `Γ` once `Γ ⊢ φ`. -/
theorem union_singleton_derivable {Γ : Set Sentence} {φ : Sentence} (h : Prov Γ φ) :
    ∀ θ ∈ Γ ∪ {φ}, Prov Γ θ := fun θ hθ =>
  Or.elim hθ (fun hΓ => assumption hΓ) (fun e => (show θ = φ from e) ▸ h)

/-- Theorem 1(c), membership form. -/
theorem mem_Cn_union_singleton_iff {Γ : Set Sentence} {φ : Sentence} (h : Prov Γ φ)
    (ψ : Sentence) : ψ ∈ Cn (Γ ∪ {φ}) ↔ ψ ∈ Cn Γ :=
  ⟨fun hψ => cut (union_singleton_derivable h) hψ,
   fun hψ => monotonicity (fun _ hx => Or.inl hx) hψ⟩

/-- Theorem 1(c), equality form: reusing a derived sentence as a premise is conservative. -/
theorem proved_as_premise_is_conservative {Γ : Set Sentence} {φ : Sentence} (h : Prov Γ φ) :
    Cn (Γ ∪ {φ}) = Cn Γ :=
  Set.ext (mem_Cn_union_singleton_iff h)

/-- Theorem 1(c), with `insert` (the form used in the original Lean sketch). -/
theorem proved_as_premise_is_conservative_insert {Γ : Set Sentence} {φ : Sentence}
    (h : Prov Γ φ) : Cn (insert φ Γ) = Cn Γ :=
  Set.ext fun _ =>
    ⟨fun hψ => cut (fun θ hθ => Or.elim hθ (fun e => (show θ = φ from e) ▸ h)
      (fun hΓ => assumption hΓ)) hψ,
     fun hψ => monotonicity (fun _ hx => Or.inr hx) hψ⟩
```

Kernel report: `union_singleton_derivable`: none; `mem_Cn_union_singleton_iff`: none; `proved_as_premise_is_conservative`: propext, Quot.sound; `proved_as_premise_is_conservative_insert`: propext, Quot.sound

The last declaration is the form used in the original Lean sketch, with `insert ψ Γ` in place of $\Gamma \cup \{\psi\}$.

3. The exact relation between "given" and "known"

Theorem 1 gives a precise formal sense in which a proved proposition can subsequently function as a given. Initially, Γ contains what is explicitly assumed. After deduction, $\mathrm{Cn}(\Gamma)$ contains what follows. But if $\psi \in \mathrm{Cn}(\Gamma)$, then reintroducing ψ as a premise changes nothing: $\mathrm{Cn}(\Gamma \cup \{\psi\}) = \mathrm{Cn}(\Gamma)$ (Theorem 1(c)). Thus established results may be compressed and reused.

given assumptions → deduction → established proposition → reuse of that proposition as a premise → no increase in deductive closure.

This is not an informal convention. It is cut. What must not be erased is the derivational scope under which the proposition entered the closure.

That last sentence is itself a theorem. Call an *entry* a pair (Γ, ψ) of a scope and a sentence.

Proposition 3.1 (A sentence does not carry its scope) Proved from reused. *(a) If two entries (Γ_1, ψ) and (Γ_2, ψ) carry the same sentence, with $\Gamma_1 \vdash \psi$ and $\Gamma_2 \nvdash \psi$, then no function of the bare sentence decides, for every entry, whether its sentence is derivable from its scope. (b) A function of the whole entry does. (c) The hypotheses of (a) are met: with $\Gamma \vdash \varphi$ exactly when $\varphi \in \Gamma$, take $\Gamma_1 = \{p\}$ and $\Gamma_2 = \varnothing$. (d) Once two premise sets are possible, a sentence does not recover the premise set it was established from. (e) Carrying the premise set with the sentence recovers it.*

Proof. (a) A function of ψ alone gives one answer on both entries, and the correct answers differ. This is the universal recovery closure of the Adjudication Seam core [18] (`universal_class_is`

_closed), applied to the readout $(\Gamma, \psi) \mapsto \psi$ and the target $(\Gamma, \psi) \mapsto (\Gamma \vdash \psi)$. (b) The identity readout recovers every target (identity_readout_recovers). (c) $p \in \{p\}$ and $p \notin \varnothing$. (d) and (e) are Part III's scope_forgotten_is_not_recoverable and scope_retained_is_recoverable [16], with sentences as claims and premise sets as scopes. □

```
/-- An entry: a scope (a premise set) together with a sentence. -/
abbrev Entry (Sentence : Type u) := Set Sentence × Sentence

/-- Proposition 3.1(a). If two entries carry the same sentence, derivable from the scope of the
first and not from the scope of the second, then no function of the bare sentence recovers whether
an entry's sentence is derivable from its scope (R8.1 `universal_class_is_closed`). -/
theorem scope_erasure_blocks_recovery (e₁ e₂ : Entry Sentence) (same : e₁.2 = e₂.2)
    (h₁ : Prov e₁.1 e₁.2) (h₂ : ¬ Prov e₂.1 e₂.2) :
    ¬ ClosureConsequences.Recovers (fun e : Entry Sentence => e.2) (fun e => Prov e.1 e.2) :=
  ClosureConsequences.universal_class_is_closed _ _ e₁ e₂ same fun h => h₂ (cast h h₁)

/-- Proposition 3.1(b). Carrying the scope with the sentence recovers it
(R8.1 `identity_readout_recovers`). -/
theorem scope_retention_recovers :
    ClosureConsequences.Recovers (fun e : Entry Sentence => e) (fun e => Prov e.1 e.2) :=
  ClosureConsequences.identity_readout_recovers _

/-- Proposition 3.1(c). The hypotheses of (a) are met: in `Two`, `p` is derivable from `{p}` and
not from `∅`. -/
theorem scope_erasure_instance :
    ¬ ClosureConsequences.Recovers (fun e : Entry Countermodels.Two => e.2)
      (fun e => Prov e.1 e.2) :=
  scope_erasure_blocks_recovery ({Countermodels.Two.p}, Countermodels.Two.p)
    (∅, Countermodels.Two.p) rfl (show Countermodels.Two.p = Countermodels.Two.p from rfl)
    (Set.notMem_empty _)

/-- Proposition 3.1(d). Once two premise sets are possible, a sentence does not recover the
premise set it was established from (Part III `scope_forgotten_is_not_recoverable`). -/
theorem scope_not_recovered_from_sentence {Sentence : Type u} (ψ : Sentence)
    {Γ₁ Γ₂ : Set Sentence} (h : Γ₁ ≠ Γ₂) :
    ¬ LambdaIII.FactorsThrough (fun c : LambdaIII.ScopedClaim Sentence (Set Sentence) => c.prop)
      (fun c => c.scope) :=
  LambdaIII.scope_forgotten_is_not_recoverable ψ h

/-- Proposition 3.1(e). Carrying the premise set with the sentence recovers it (Part III
`scope_retained_is_recoverable`). -/
theorem scope_recovered_when_carried {Sentence : Type u} :
    LambdaIII.FactorsThrough
      (fun c : LambdaIII.ScopedClaim Sentence (Set Sentence) => (c.prop, c.scope))
      (fun c => c.scope) :=
  LambdaIII.scope_retained_is_recoverable
```

Kernel report: scope_erasure_blocks_recovery: none; scope_retention_recovers: none; scope_erasure_instance: none; scope_not_recovered_from_sentence: none; scope_recovered_when_carried: none

Part III states (d) and (e) for editorial operations, and R4 checks the same pattern for the origin of a claim: a result label does not determine whether the claim was declared or derived (result_label_cannot_recover_origin) Reused.

R4 records an established result as a *warrant*: a scope, a claim, and a receipt that the claim holds in that scope. Unchanged reuse needs the same scope (warranted_reuse_needs_scope_preservation); moving to another scope or claim needs a bridge (warranted_translation_needs_a_bridge); and a tag alone proves nothing (derived_tag_alone_does_not_prove_claim) Reused. A

derivation $\Gamma \vdash \psi$ is such a warrant, with scope Γ and claim ψ, and the two halves of Theorem 1 are bridges.

Proposition 3.2 (Reuse and translation are warrant operations) Proved from reused. *(a) Enlarging the premise set translates every warrant: if $\Gamma \vdash \psi$, then $\Gamma \cup \Delta \vdash \psi$, with monotonicity (Lemma 1.2) as the bridge. (b) Discharging an established premise translates every warrant: if $\Gamma \vdash \psi$ and $\Gamma \cup \{\psi\} \vdash \theta$, then $\Gamma \vdash \theta$, with Theorem 1(c) as the bridge.*

Proof. Both are R4's `warranted_translation_needs_a_bridge`. For (a) the scope map is $\Gamma \mapsto \Gamma \cup \Delta$ and the bridge is Lemma 1.2. For (b) the scopes are the premise sets that derive ψ, a warrant at such a scope Γ certifies a sentence derivable from $\Gamma \cup \{\psi\}$, the scope map returns Γ, and the bridge is Theorem 1(c). □

```
theorem warranted_translation_needs_a_bridge {S T C D : Type*}
    {holds : S → C → Prop} {nextHolds : T → D → Prop}
    (scopeMap : S → T) (claimMap : C → D)
    (bridge : ∀ s c, holds s c → nextHolds (scopeMap s) (claimMap c))
    (w : Warrant S C holds) : nextHolds (scopeMap w.scope) (claimMap w.claim) := ...
```

Kernel report: `warranted_translation_needs_a_bridge`: none

```
/-- A derivation `Γ ⊢ ψ` as a warrant in the sense of R4: scope `Γ`, claim `ψ`, receipt the
derivation itself. -/
def derivationWarrant {Γ : Set Sentence} {ψ : Sentence} (h : Prov Γ ψ) :
    LambdaConstitutive.Warrant (Set Sentence) Sentence Prov := ⟨Γ, ψ, h⟩

/-- Proposition 3.2(a). Enlarging the premise set translates every warrant; monotonicity
(Lemma 1.2) is the bridge (R4 `warranted_translation_needs_a_bridge`). -/
theorem enlarging_scope_translates (Δ : Set Sentence)
    (w : LambdaConstitutive.Warrant (Set Sentence) Sentence Prov) :
    Prov (w.scope ∪ Δ) w.claim :=
  LambdaConstitutive.warranted_translation_needs_a_bridge (nextHolds := Prov)
    (fun s => s ∪ Δ) id (fun _ _ h => monotonicity Set.subset_union_left h) w

/-- Proposition 3.2(b). Discharging an established premise translates every warrant; Theorem 1(c)
is the bridge (R4 `warranted_translation_needs_a_bridge`). -/
theorem discharging_established_premise_translates (ψ : Sentence)
    (w : LambdaConstitutive.Warrant {Γ : Set Sentence // Prov Γ ψ} Sentence
      (fun Γ θ => Prov (Γ.1 ∪ {ψ}) θ)) :
    Prov w.scope.1 w.claim :=
  LambdaConstitutive.warranted_translation_needs_a_bridge (nextHolds := Prov)
    Subtype.val id (fun Γ _ h => (mem_Cn_union_singleton_iff Γ.2 _).1 h) w
```

Kernel report: `enlarging_scope_translates`: none; `discharging_established_premise_translates`: none

4. Symmetric Refutation Theorem

Definition 4.1 (Consistency). Γ is consistent if there is no sentence φ for which both $\Gamma \vdash \varphi$ and $\Gamma \vdash \neg\varphi$.

Definition 4.2 (Contradictory pair). Sentences α, β are contradictory if $\beta = \neg\alpha$ or $\alpha = \neg\beta$.

Definition 4.2 is symmetric in α and β. It is introduced so that the same statements apply to a claim and to an objection with their roles exchanged (Theorem 6(c)).

```
/-- Consistency: no sentence is derived together with its negation. -/
def Consistent (Γ : Set Sentence) : Prop := ∀ φ, ¬ (Prov Γ φ ∧ Prov Γ (neg φ))

/-- `α` and `β` are contradictory: one is the negation of the other. -/
def Contradictory (α β : Sentence) : Prop := β = neg α ∨ α = neg β

theorem Contradictory.symm {α β : Sentence} (h : Contradictory α β) : Contradictory β α :=
  Or.elim h Or.inr Or.inl

theorem contradictory_neg (φ : Sentence) : Contradictory φ (neg φ) := Or.inl rfl
```

Kernel report: `symm`: none; `contradictory_neg`: none

Lemma 4.3 Proved**.** *A consistent* Γ *derives no contradictory pair. In particular, if* Γ *is consistent and* $\Gamma \vdash \varphi$*, then* $\Gamma \nvdash \neg\varphi$*.*

Proof. If $\beta = \neg\alpha$, then $\Gamma \vdash \alpha$ and $\Gamma \vdash \neg\alpha$; if $\alpha = \neg\beta$, then $\Gamma \vdash \beta$ and $\Gamma \vdash \neg\beta$. Either way consistency fails. □

```
/-- A consistent set derives no contradictory pair. -/
theorem Consistent.not_both {Γ : Set Sentence} (hc : Consistent Γ) {α β : Sentence}
    (h : Contradictory α β) : ¬ (Prov Γ α ∧ Prov Γ β) := fun ⟨hα, hβ⟩ =>
  Or.elim h (fun e => hc α ⟨hα, e ▸ hβ⟩) (fun e => hc β ⟨hβ, e ▸ hα⟩)

/-- A consistent set that derives `φ` does not derive `¬φ`. -/
theorem Consistent.not_prov_neg {Γ : Set Sentence} (hc : Consistent Γ) {φ : Sentence}
    (h : Prov Γ φ) : ¬ Prov Γ (neg φ) := fun hn => hc φ ⟨h, hn⟩
```

Kernel report: `not_both`: none; `not_prov_neg`: none

Theorem 2 (A refutation under the same standard requires either a defect or a genuinely new premise) Proved**.** *Assume* $\Gamma \vdash \varphi$ *and that* Γ *is consistent. Suppose some argument derives* $\Gamma \cup \Delta \vdash \neg\varphi$*. Then:*

(a) not every member of Δ *is derivable from* Γ*;*

(b) at least one member of Δ *is not derivable from* Γ*: there is* $\delta \in \Delta$ *with* $\Gamma \nvdash \delta$*.*

More generally, (a) holds with $\varphi, \neg\varphi$ *replaced by any contradictory pair* α, β*: if* $\Gamma \vdash \alpha$*,* Γ *is consistent and* $\Gamma \cup \Delta \vdash \beta$*, then not every member of* Δ *is derivable from* Γ*.*

Proof. (a) Assume for contradiction that every member of Δ is derivable from Γ. Every member of Γ is also derivable from Γ, by the assumption rule. Hence every premise in $\Gamma \cup \Delta$ is derivable from Γ. But $\Gamma \cup \Delta \vdash \neg\varphi$; by cut, $\Gamma \vdash \neg\varphi$. By hypothesis $\Gamma \vdash \varphi$. Therefore Γ proves both φ and its negation, contrary to consistency. The general form is the same argument with Lemma 4.3.

(b) follows from (a) by classical logic: the negation of a universal statement yields an instance of the negated property. □

Statement (a) is the constructive content of the theorem. Statement (b) is the original existential form; the passage from (a) to (b) is its only non-constructive step, and the kernel reports below show it.

```
/-- Theorem 2, general constructive core: if `Γ ⊢ α`, `Γ` is consistent and `Γ ∪ Δ ⊢ β` for a
sentence `β` contradicting `α`, then not every member of `Δ` is derivable from `Γ`. -/
theorem not_all_premises_derivable {Γ Δ : Set Sentence} {α β : Sentence}
```

```
    (hα : Prov Γ α) (hcons : Consistent Γ) (hc : Contradictory α β)
    (hβ : Prov (Γ ∪ Δ) β) : ¬ ∀ δ ∈ Δ, Prov Γ δ := fun hall =>
  hcons.not_both hc
    ⟨hα, cut (fun θ hθ => Or.elim hθ (fun h => assumption h) (fun h => hall θ h)) hβ⟩

/-- Theorem 2, constructive core. -/
theorem refutation_requires_new_premise {Γ Δ : Set Sentence} {φ : Sentence}
    (hpos : Prov Γ φ) (hcons : Consistent Γ) (hneg : Prov (Γ ∪ Δ) (neg φ)) :
    ¬ ∀ δ ∈ Δ, Prov Γ δ :=
  not_all_premises_derivable hpos hcons (contradictory_neg φ) hneg

/-- Theorem 2, existential form (classical). -/
theorem negative_derivation_requires_new_premise {Γ Δ : Set Sentence} {φ : Sentence}
    (hpos : Prov Γ φ) (hcons : Consistent Γ) (hneg : Prov (Γ ∪ Δ) (neg φ)) :
    ∃ δ ∈ Δ, ¬ Prov Γ δ := by
  by_contra h
  exact refutation_requires_new_premise hpos hcons hneg
    (fun δ hδ => by_contra fun hn => h ⟨δ, hδ, hn⟩)
```

Kernel report: `not_all_premises_derivable`: none; `refutation_requires_new_premise`: none; `negative_derivation_requires_new_premise`: propext, Classical.choice, Quot.sound

Theorem 2′ (The refuting premise set is itself inconsistent) Proved**.** *If* $\Gamma \vdash \varphi$ *and* $\Gamma \cup \Delta \vdash \neg\varphi$*, then* $\Gamma \cup \Delta$ *is inconsistent. No consistency hypothesis on* Γ *is needed.*

Proof. By monotonicity, $\Gamma \cup \Delta \vdash \varphi$. Together with $\Gamma \cup \Delta \vdash \neg\varphi$ this contradicts consistency of $\Gamma \cup \Delta$. □

Thus an argument that retains the established premises and derives the negation of an established consequence has an inconsistent premise set.

```
/-- Theorem 2(b): a premise set that keeps `Γ` and derives `¬φ` is itself inconsistent
whenever `Γ ⊢ φ`. No consistency hypothesis is needed. -/
theorem refuting_extension_inconsistent {Γ Δ : Set Sentence} {φ : Sentence}
    (hpos : Prov Γ φ) (hneg : Prov (Γ ∪ Δ) (neg φ)) : ¬ Consistent (Γ ∪ Δ) := fun hc =>
  hc φ (monotonicity (fun _ h => Or.inl h) hpos, hneg)
```

Kernel report: `refuting_extension_inconsistent`: none

Proposition 4.4 (Both hypotheses of Theorem 2 are needed) Proved**.**

(i) Consistency. *On a one-sentence set with the total relation (every sentence derivable from every set; assumption and cut hold), take* $\Gamma = \Delta = \varnothing$*. Then* $\Gamma \vdash \varphi$ *and* $\Gamma \cup \Delta \vdash \neg\varphi$*, while every member of* Δ *is derivable from* Γ*; here* Γ *is inconsistent.*

(ii) The derivation $\Gamma \vdash \varphi$. *On the two sentences* $p, \neg p$ *with* $\Gamma \vdash \varphi$ *iff* $\varphi \in \Gamma$ *(assumption and cut hold), take* $\Gamma = \{\neg p\}$*,* $\Delta = \varnothing$*,* $\varphi = p$*. Then* Γ *is consistent,* $\Gamma \nvdash p$*, and* $\Gamma \cup \Delta \vdash \neg p$*, while every member of* Δ *is derivable from* Γ*.*

```
/-- Two sentences, `p` and its negation `np`. -/
inductive Two where
  | p
  | np

/-- On `Two`, `Γ ⊢ φ` holds exactly when `φ ∈ Γ`. -/
instance twoConsequence : Consequence Two where
```

```
  Prov Γ φ := φ ∈ Γ
  assumption h := h
  cut hΔ hφ := hΔ _ hφ
  neg
    | .p => .np
    | .np => .p

/-- A one-sentence language in which everything is derivable from everything. -/
inductive One where
  | s

instance oneConsequence : Consequence One where
  Prov _ _ := True
  assumption _ := trivial
  cut _ _ := trivial
  neg _ := .s

/-- Theorem 2 needs consistency. -/
theorem refutation_needs_consistency :
    ∃ (Γ Δ : Set One) (φ : One), Prov Γ φ ∧ Prov (Γ ∪ Δ) (neg φ) ∧ (∀ δ ∈ Δ, Prov Γ δ) ∧
      ¬ Consistent Γ :=
  ⟨∅, ∅, .s, trivial, trivial, fun _ _ => trivial, fun h => h .s ⟨trivial, trivial⟩⟩

/-- Theorem 2 needs `Γ ⊢ φ`. -/
theorem refutation_needs_derivation :
    ∃ (Γ Δ : Set Two) (φ : Two), Consistent Γ ∧ ¬ Prov Γ φ ∧ Prov (Γ ∪ Δ) (neg φ) ∧
      (∀ δ ∈ Δ, Prov Γ δ) := by
  refine ⟨{Two.np}, ∅, Two.p, ?_, ?_, ?_, ?_⟩
  · intro ψ ⟨h1, h2⟩
    cases ψ with
    | p => exact Two.noConfusion (show Two.p = Two.np from h1)
    | np => exact Two.noConfusion (show Two.p = Two.np from h2)
  · intro h
    exact Two.noConfusion (show Two.p = Two.np from h)
  · exact Or.inl (show Two.np = Two.np from rfl)
  · intro δ hδ
    exact absurd hδ (Set.notMem_empty δ)
```

Kernel report: `refutation_needs_consistency`: none; `refutation_needs_derivation`: none

5. Consequence for adjudication

Theorem 2 gives a structural classification of a purported refutation of an established result under a fixed proof standard. If $\Gamma \vdash \varphi$, then a genuine challenge must do at least one of the following:

1. show that an alleged premise of Γ was not actually admitted;
2. show that a purported inference in the proof is invalid;
3. show that the calculus or assumptions are inconsistent;
4. change the assumptions by introducing some δ not derivable from Γ;
5. change the interpretation of one or more terms;
6. change the consequence relation.

That these six cases are exhaustive, relative to the formal setting, is Theorem 6 in Section 12, where each case is the failure of one named hypothesis.

What cannot constitute a refutation under the same standard is merely "I do not accept φ." That sentence reports an attitude. It does not derive $\neg\varphi$. Similarly, "perhaps $\neg\varphi$" does not establish $\neg\varphi$. If the negative claim is to be promoted under the same proof standard, it requires its own adequate derivation. The following statements make this exact. An attitude is represented by an arbitrary predicate on sentences; no rule of the consequence relation mentions it.

Proposition 5.1 (Non-acceptance is not refutation) Proved. *Suppose* $\Gamma \vdash \varphi$ *and* Γ *is consistent. Then there is no rule "if* Accept(φ) *fails, then* $\Gamma \vdash \neg\varphi$*" that holds for every predicate* Accept*.*

Proof. Take Accept to be identically false. Such a rule would give $\Gamma \vdash \neg\varphi$, contradicting Lemma 4.3. □

Proposition 5.2 (A hedge is not a derivation) Proved. *Suppose* $\Gamma \vdash \varphi$ *and* Γ *is consistent. Then there is no rule "if* Perhaps$(\neg\varphi)$*, then* $\Gamma \vdash \neg\varphi$*" that holds for every predicate* Perhaps*.*

Proof. Take Perhaps to be identically true and argue as in Proposition 5.1. □

Proposition 5.3 (A rule from non-acceptance to refutation constrains the adjudicator) Proved. *Let* Accept *be a predicate, and suppose the rule "*$\neg$Accept$(\psi) \Rightarrow \Gamma \vdash \neg\psi$*" holds for every* ψ*. If* $\Gamma \vdash \varphi$ *and* Γ *is consistent, then* Accept(φ) *cannot fail:* $\neg\neg$Accept(φ)*.*

Proof. If Accept(φ) failed, the rule would give $\Gamma \vdash \neg\varphi$, contradicting Lemma 4.3. □

The adjudicator role supplies no inference rule. A rule connecting non-acceptance to derivation is an addition to the consequence relation (case 6), and in a consistent setting such a rule obliges acceptance of every derived sentence.

Proposition 5.4 (Not deriving φ is not deriving $\neg\varphi$) Proved. *The rule "if* $\Gamma \nvdash \varphi$ *then* $\Gamma \vdash \neg\varphi$*" fails: on the two sentences* $p, \neg p$ *with* $\Gamma \vdash \varphi$ *iff* $\varphi \in \Gamma$*, the empty set derives neither* p *nor* $\neg p$*.*

```
/-- Non-acceptance is not refutation: in a consistent setting with `Γ ⊢ φ`, no rule of the form
"if `φ` is not accepted then `Γ ⊢ ¬φ`" can hold for every acceptance predicate. -/
theorem nonacceptance_not_refutation {Γ : Set Sentence} {φ : Sentence}
    (hpos : Prov Γ φ) (hcons : Consistent Γ) :
    ¬ ∀ Accept : Sentence → Prop, ¬ Accept φ → Prov Γ (neg φ) :=
  fun h => hcons φ ⟨hpos, h (fun _ => False) id⟩

/-- A hedge is not a derivation: no rule "if perhaps `¬φ` then `Γ ⊢ ¬φ`" holds for every
hedging predicate, in a consistent setting with `Γ ⊢ φ`. -/
theorem hedge_not_refutation {Γ : Set Sentence} {φ : Sentence}
    (hpos : Prov Γ φ) (hcons : Consistent Γ) :
    ¬ ∀ Perhaps : Sentence → Prop, Perhaps (neg φ) → Prov Γ (neg φ) :=
  fun h => hcons φ ⟨hpos, h (fun _ => True) trivial⟩

/-- If non-acceptance were adjoined as an inference rule, consistency would force acceptance
of every derived sentence. -/
theorem bridge_rule_forces_acceptance {Γ : Set Sentence} {φ : Sentence}
    (Accept : Sentence → Prop) (bridge : ∀ ψ, ¬ Accept ψ → Prov Γ (neg ψ))
    (hpos : Prov Γ φ) (hcons : Consistent Γ) : ¬ ¬ Accept φ :=
  fun hna => hcons φ ⟨hpos, bridge φ hna⟩
```

```
/-- Not deriving `φ` is not deriving `¬φ`. -/
theorem underivable_not_refuted : ¬ ∀ (Γ : Set Two) (φ : Two), ¬ Prov Γ φ → Prov Γ (neg φ) :=
  fun h => Set.notMem_empty _ (h ∅ Two.p (Set.notMem_empty _))
```

Kernel report: `nonacceptance_not_refutation`: none; `hedge_not_refutation`: none; `bridge_rule_forces_acceptance`: none; `underivable_not_refuted`: none

This is symmetry of standard, not symmetry of certificate shape. An existential, a universal, and a negated universal can require different forms of evidence because their logical forms differ.

The same point has earlier forms in the corpus. The Double-Edged Receipt Standard [20] states it as a corollary: a demand for warrant may block promotion of a claim but supplies no warrant for its negation, and the warranted status may be OPEN rather than FALSE (written proof). R4 checks that the absence of a certified refutation is not a general proof (`no_refutation_is_not_a_general_proof`) and that completeness at the negation is the premise that licenses the reversal (`completeness_licenses_reversal`) Reused. A semantic counterpart of Proposition 5.4, from the R6 diagonal certificate, is Corollary 19.2(c).

```
theorem no_refutation_is_not_a_general_proof :
    ¬ (∀ (certified : Prop → Prop) (P : Prop),
      (∀ Q, certified Q → Q) → ¬ certified (¬ P) → P) := ...

theorem completeness_licenses_reversal
    (certified : Prop → Prop) (P : Prop)
    (completeAtNegation : (¬ P) → certified (¬ P))
    (noRefutation : ¬ certified (¬ P)) : P := ...
```

Kernel report: `no_refutation_is_not_a_general_proof`: none; `completeness_licenses_reversal`: propext, Classical.choice, Quot.sound

6. Class-Level Uncertainty

There are two notions that must be kept distinct.

6.1 Operational openness

At a particular stage of inquiry, we may simply possess neither an accepted derivation of φ nor one of $\neg\varphi$. This is a state of the proof registry.

Definition 6.1 (Registry; openness). A registry R is a predicate on sentences: $R(\psi)$ means that ψ currently has an accepted receipt (an accepted certificate). Write $\mathrm{Open}_R(\varphi)$ when neither $R(\varphi)$ nor $R(\neg\varphi)$. A registry is sound for Γ when $R(\psi)$ implies $\Gamma \vdash \psi$.

$\mathrm{Open}_R(\varphi)$ means only that no presently accepted receipt closes φ and none closes $\neg\varphi$. It does not itself prove that neither derivation exists.

6.2 Formal independence

Definition 6.2 (Independence). $\mathrm{Indep}_\Gamma(\varphi)$ means $\Gamma \nvdash \varphi$ and $\Gamma \nvdash \neg\varphi$.

This is a metamathematical statement about the entire deductive class generated by Γ. Gödel–Rosser incompleteness provides such statements for broad classes of sufficiently expressive consistent effective theories (Section 19).

```
/-- A registry records which sentences currently have an accepted receipt. -/
structure Registry (Sentence : Type u) where
  receipt : Sentence → Prop

/-- Operational openness: neither `φ` nor `¬φ` has an accepted receipt. -/
def Registry.Open (R : Registry Sentence) (φ : Sentence) : Prop :=
  ¬ R.receipt φ ∧ ¬ R.receipt (neg φ)

/-- A registry is sound for `Γ` when every receipt is backed by a derivation from `Γ`. -/
def Registry.SoundFor (R : Registry Sentence) (Γ : Set Sentence) : Prop :=
  ∀ ψ, R.receipt ψ → Prov Γ ψ

/-- Formal independence: `Γ` derives neither `φ` nor `¬φ`. -/
def Indep (Γ : Set Sentence) (φ : Sentence) : Prop := ¬ Prov Γ φ ∧ ¬ Prov Γ (neg φ)
```

Proposition 6.3 (Independence implies openness) Proved**.** *If R is sound for Γ and* $\mathrm{Indep}_\Gamma(\varphi)$*, then* $\mathrm{Open}_R(\varphi)$*.*

Proof. A receipt for φ or for $\neg\varphi$ would give a derivation from Γ. □

Proposition 6.4 (Openness does not imply independence) Proved**.** *There are a registry R sound for Γ, and a sentence φ, with* $\mathrm{Open}_R(\varphi)$ *but not* $\mathrm{Indep}_\Gamma(\varphi)$*.*

Proof. On the two sentences $p, \neg p$ with $\Gamma \vdash \varphi$ iff $\varphi \in \Gamma$, take $\Gamma = \{p\}$ and the registry with no receipts. It is sound for Γ and open on p, but $\Gamma \vdash p$. □

```
/-- Independence implies openness in every registry sound for `Γ`. -/
theorem Registry.open_of_indep {R : Registry Sentence} {Γ : Set Sentence} (hR : R.SoundFor Γ)
    {φ : Sentence} (h : Indep Γ φ) : R.Open φ :=
  ⟨fun r => h.1 (hR φ r), fun r => h.2 (hR _ r)⟩

/-- Operational openness does not imply independence. -/
theorem open_not_indep :
    ∃ (R : Registry Two) (Γ : Set Two) (φ : Two), R.SoundFor Γ ∧ R.Open φ ∧ ¬ Indep Γ φ :=
  ⟨⟨fun _ => False⟩, {Two.p}, Two.p, fun _ h => h.elim, ⟨id, id⟩,
    fun h => h.1 (show Two.p = Two.p from rfl)⟩
```

Kernel report: `open_of_indep`: none; `open_not_indep`: none

So a registry state of openness and formal independence are different statements, and neither is promoted to the other without a theorem connecting them in the setting at hand.

7. Model-theoretic form of class-level uncertainty

Now specialize to classical first-order logic and use its soundness and completeness theorems. The statements below are proved for any consequence relation and any semantics satisfying the hypotheses named in Definition 7.1; classical first-order logic is the instance at issue.

Definition 7.1 (Semantics). A semantics is a class of models with a satisfaction relation $M \models \psi$. Write $M \models \Gamma$ if $M \models \psi$ for all $\psi \in \Gamma$, and $\Gamma \models \varphi$ if every model of Γ satisfies φ. The relation $\vdash$ is *complete* if $\Gamma \models \varphi$ implies $\Gamma \vdash \varphi$, and *sound* if $\Gamma \vdash \varphi$ implies $\Gamma \models \varphi$. The semantics obeys the *Tarski clause* if $M \models \neg\varphi$ iff $M \not\models \varphi$.

For classical first-order logic, completeness and soundness are Gödel's completeness theorem and the soundness theorem Premise; the Tarski clause is the definition of satisfaction for negation. In

the Lean statements, completeness and soundness are the named hypotheses `hcomp : Complete Sat` and `hsound : Sound Sat`.

```
variable {Model : Type v} (Sat : Model → Sentence → Prop)

/-- `M` satisfies every member of `Γ`. -/
def ModelsAll (M : Model) (Γ : Set Sentence) : Prop := ∀ ψ ∈ Γ, Sat M ψ

/-- Semantic consequence. -/
def Entails (Γ : Set Sentence) (φ : Sentence) : Prop := ∀ M, ModelsAll Sat M Γ → Sat M φ

/-- Completeness of `⊢` for the semantics (Gödel 1930 for classical first-order logic). -/
def Complete : Prop := ∀ (Γ : Set Sentence) (φ : Sentence), Entails Sat Γ φ → Prov Γ φ

/-- Soundness of `⊢` for the semantics. -/
def Sound : Prop := ∀ (Γ : Set Sentence) (φ : Sentence), Prov Γ φ → Entails Sat Γ φ

/-- The classical (Tarski) clause for negation. -/
def NegClause : Prop := ∀ (M : Model) (φ : Sentence), Sat M (neg φ) ↔ ¬ Sat M φ
```

Theorem 3 (Model split for an independent sentence) Proved from premise. *Let $\vdash$ be complete, and let the semantics obey the Tarski clause. If $\Gamma \nvdash \varphi$ and $\Gamma \nvdash \neg\varphi$, then:*

(a) not every model of Γ satisfies $\neg\varphi$, and not every model of Γ satisfies φ;

(b) there exist models M and N such that $M \models \Gamma$, $M \models \varphi$, while $N \models \Gamma$, $N \models \neg\varphi$.

Proof. (a) If every model of Γ satisfied $\neg\varphi$, then $\Gamma \models \neg\varphi$, and completeness would give $\Gamma \vdash \neg\varphi$, contrary to hypothesis. Likewise, if every model of Γ satisfied φ, completeness would give $\Gamma \vdash \varphi$.

(b) Suppose there were no model of $\Gamma \cup \{\varphi\}$. Then every model of Γ fails φ and so, by the Tarski clause, satisfies $\neg\varphi$; that is, $\Gamma \models \neg\varphi$. By completeness, $\Gamma \vdash \neg\varphi$, contrary to hypothesis. Therefore there exists a model M with $M \models \Gamma \cup \{\varphi\}$. Similarly, if no model satisfied $\Gamma \cup \{\neg\varphi\}$, then every model of Γ fails $\neg\varphi$, hence satisfies φ, so $\Gamma \models \varphi$ and completeness would yield $\Gamma \vdash \varphi$, again contradicting the hypothesis. Therefore there exists N satisfying $\Gamma \cup \{\neg\varphi\}$. □

Part (a) is constructive. Part (b) uses classical logic twice: to pass from "not no model" to a model, and, in its second half, from "not $M \models \neg\varphi$" to $M \models \varphi$.

```
/-- Theorem 3, constructive core (first half): not every model of `Γ` satisfies `¬φ`. -/
theorem model_split_core_pos (hcomp : Complete Sat) {Γ : Set Sentence} {φ : Sentence}
    (hn : ¬ Prov Γ (neg φ)) : ¬ ∀ M, ModelsAll Sat M Γ → Sat M (neg φ) :=
  fun h => hn (hcomp Γ (neg φ) h)

/-- Theorem 3, constructive core (second half): not every model of `Γ` satisfies `φ`. -/
theorem model_split_core_neg (hcomp : Complete Sat) {Γ : Set Sentence} {φ : Sentence}
    (hp : ¬ Prov Γ φ) : ¬ ∀ M, ModelsAll Sat M Γ → Sat M φ :=
  fun h => hp (hcomp Γ φ h)

/-- Theorem 3 (classical): an independent sentence holds in some model of `Γ` and fails in
another. Consistency of `Γ` is not assumed. -/
theorem model_split (hcomp : Complete Sat) (hneg : NegClause Sat) {Γ : Set Sentence}
    {φ : Sentence} (hind : Indep Γ φ) :
    (∃ M, ModelsAll Sat M Γ ∧ Sat M φ) ∧ (∃ N, ModelsAll Sat N Γ ∧ Sat N (neg φ)) := by
  constructor
  · by_contra h
    exact model_split_core_pos hcomp hind.2 fun M hM => (hneg M φ).2 fun hφ => h ⟨M, hM, hφ⟩
```

```
  · by_contra h
    exact model_split_core_neg hcomp hind.1 fun M hM =>
      by_contra fun hφ => h ⟨M, hM, (hneg M φ).2 hφ⟩
```

Kernel report: `model_split_core_pos`: none; `model_split_core_neg`: none; `model_split`: propext, Classical.choice, Quot.sound

The original statement of Theorem 3 assumes in addition that Γ is consistent. The proof does not use that assumption, so it is removed. In a calculus with explosion, as in classical first-order logic, independence already implies consistency:

Lemma 7.2 Proved. *If every sentence is derivable from any set that derives some sentence together with its negation, then* $\mathrm{Indep}_\Gamma(\varphi)$ *implies that* Γ *is consistent.*

```
/-- Explosion: an inconsistent premise set derives every sentence. -/
def Explosive (Sentence : Type u) [Consequence Sentence] : Prop :=
  ∀ (Γ : Set Sentence) (φ ψ : Sentence), Prov Γ φ → Prov Γ (neg φ) → Prov Γ ψ

/-- Under explosion, an independent sentence certifies consistency. -/
theorem consistent_of_indep (hexp : Explosive Sentence) {Γ : Set Sentence} {φ : Sentence}
    (h : Indep Γ φ) : Consistent Γ :=
  fun ψ ⟨h1, h2⟩ => h.1 (hexp Γ ψ φ h1 h2)
```

Kernel report: `consistent_of_indep`: none

Lemma 7.3 (A countermodel establishes underivability) Proved from premise. *If* $\vdash$ *is sound,* $M \models \Gamma$ *and* $M \not\models \varphi$*, then* $\Gamma \nvdash \varphi$*.*

```
/-- A countermodel establishes underivability, given soundness. -/
theorem not_prov_of_countermodel (hsound : Sound Sat) {Γ : Set Sentence} {φ : Sentence}
    {M : Model} (hM : ModelsAll Sat M Γ) (hφ : ¬ Sat M φ) : ¬ Prov Γ φ :=
  fun h => hφ (hsound Γ φ h M hM)
```

Kernel report: `not_prov_of_countermodel`: none

This is not a third truth value. It is a class-level fact about what the fixed assumptions determine.

Theorem 3 has a reading in terms of adjudication. What Γ settles about φ is the pair of propositions $\Gamma \vdash \varphi$ and $\Gamma \vdash \neg\varphi$; the pair is the same whichever model of Γ is considered.

Theorem 7.4 (Independence as an adjudication seam) Proved from reused; premise. *Let* $\vdash$ *be complete for a semantics obeying the Tarski clause, and let* $\Gamma \nvdash \varphi$ *and* $\Gamma \nvdash \neg\varphi$*. Then:*

(a) there are models M *and* N *of* Γ *on which what* Γ *settles about* φ *is the same while the truth values of* φ *differ;*

(b) no function of what Γ *settles recovers the truth value of* φ *on every model of* Γ*;*

(c) every sound stage-indexed adjudicator whose input is what Γ *settles returns no verdict at any stage.*

Proof. (a) Theorem 3 gives M with $M \models \varphi$ and N with $N \models \neg\varphi$, so $N \not\models \varphi$ by the Tarski clause. (b) and (c) are the universal recovery closure and the forced abstention of the Adjudication Seam core [18] (`universal_class_is_closed`, `forced_abstention_at_seam`) applied to M and N. An adjudicator is sound when every verdict it returns at any stage is the truth value at every model with that input; a verdict at the shared input would have to equal both truth values. □

```
/-- The models of `Γ`. -/
abbrev ModelOf (Sat : Model → Sentence → Prop) (Γ : Set Sentence) : Type v :=
  {M : Model // ModelsAll Sat M Γ}

/-- What `Γ` settles about `φ`: whether `Γ ⊢ φ`, and whether `Γ ⊢ ¬φ`. The record is the same
whichever model of `Γ` is considered. -/
def settles (Γ : Set Sentence) (φ : Sentence) (_ : ModelOf Sat Γ) : Prop × Prop :=
  (Prov Γ φ, Prov Γ (neg φ))

/-- Theorem 7.4(a). An independent sentence is a seam: two models of `Γ` give the same record of
what `Γ` settles and different truth values of `φ`. Completeness enters as a hypothesis, as in
Theorem 3. -/
theorem indep_is_seam (hcomp : Complete Sat) (hneg : NegClause Sat) {Γ : Set Sentence}
    {φ : Sentence} (hind : Indep Γ φ) :
    ∃ M N : ModelOf Sat Γ, settles Γ φ M = settles Γ φ N ∧ Sat M.1 φ ≠ Sat N.1 φ := by
  obtain ⟨⟨M, hM, hMφ⟩, ⟨N, hN, hNφ⟩⟩ := model_split hcomp hneg hind
  exact ⟨⟨M, hM⟩, ⟨N, hN⟩, rfl, fun h => (hneg N φ).1 hNφ (cast h hMφ)⟩

/-- Theorem 7.4(b). No adjudicator that reads only what `Γ` settles recovers the truth value of an
independent sentence across the models of `Γ` (R8.1 `universal_class_is_closed`). -/
theorem indep_truth_not_recovered (hcomp : Complete Sat) (hneg : NegClause Sat)
    {Γ : Set Sentence} {φ : Sentence} (hind : Indep Γ φ) :
    ¬ ClosureConsequences.Recovers (settles (Sat := Sat) Γ φ) (fun M => Sat M.1 φ) := by
  obtain ⟨M, N, same, diff⟩ := indep_is_seam hcomp hneg hind
  exact ClosureConsequences.universal_class_is_closed _ _ M N same diff

/-- Theorem 7.4(c). Every sound stage-indexed adjudicator that reads only what `Γ` settles abstains
at every stage on an independent sentence (R8.1 `forced_abstention_at_seam`). -/
theorem indep_forces_abstention (hcomp : Complete Sat) (hneg : NegClause Sat)
    {Γ : Set Sentence} {φ : Sentence} (hind : Indep Γ φ)
    (attempt : Prop × Prop → Nat → Option Prop)
    (sound : ClosureConsequences.SoundAttempt (settles (Sat := Sat) Γ φ) (fun M => Sat M.1 φ)
      attempt) :
    ClosureConsequences.CertifiedAbstention attempt (Prov Γ φ, Prov Γ (neg φ)) := by
  obtain ⟨M, N, same, diff⟩ := indep_is_seam hcomp hneg hind
  exact ClosureConsequences.forced_abstention_at_seam _ _ M N same diff attempt sound
```

Kernel report: `indep_is_seam`: propext, Classical.choice, Quot.sound; `indep_truth_not_recovered`: propext, Classical.choice, Quot.sound; `indep_forces_abstention`: propext, Classical.choice, Quot.sound

Part (c) is the formal content of the remark before it and of status C in Section 20: the open status belongs to an adjudication that reads only what Γ settles; it is not a third truth value of φ.

8. Undecidability is relative to the specified deductive setting

Corollary 3.1 (Independence does not mean "undecidable forever"). Proved from premise.
Let $\vdash$ be sound and complete, and let the semantics obey the Tarski clause. If $\mathrm{Indep}_\Gamma(\varphi)$*, then:*

(a) $\Gamma + \varphi$ *is consistent;*

(b) $\Gamma + \neg\varphi$ *is consistent;*

(c) $\Gamma + \varphi \vdash \varphi$ *and* $\Gamma + \neg\varphi \vdash \neg\varphi$.

Thus a sentence undecidable in Γ *becomes decided in either extension.*

Proof. (c) follows immediately from the assumption rule.

(a) Suppose $\Gamma + \varphi \vdash \psi$ and $\Gamma + \varphi \vdash \neg\psi$. By soundness every model of $\Gamma + \varphi$ satisfies both ψ and $\neg\psi$, which the Tarski clause excludes; so $\Gamma + \varphi$ has no model. Then every model of Γ fails φ and therefore satisfies $\neg\varphi$, contradicting Theorem 3(a).

(b) Theorem 3(b) gives $N \models \Gamma + \neg\varphi$. If $\Gamma + \neg\varphi$ derived some ψ and $\neg\psi$, soundness would make N satisfy both. □

The original proof says that consistency "follows from Theorem 3"; the step from a model to consistency is soundness, which Section 7 invokes and which enters here as an explicit hypothesis. The original also assumes that Γ is consistent; this is not used. Part (a) is constructive; part (b) inherits the classical step of Theorem 3(b).

```
/-- Corollary 3.1(a), constructive: `Γ ∪ {φ}` is consistent. -/
theorem indep_union_pos_consistent (hsound : Sound Sat) (hcomp : Complete Sat)
    (hneg : NegClause Sat) {Γ : Set Sentence} {φ : Sentence} (hind : Indep Γ φ) :
    Consistent (Γ ∪ {φ}) := by
  intro ψ ⟨h1, h2⟩
  refine model_split_core_pos hcomp hind.2 fun M hM => (hneg M φ).2 fun hφ => ?_
  have hM' : ModelsAll Sat M (Γ ∪ {φ}) := fun θ hθ =>
    Or.elim hθ (fun h => hM θ h) (fun e => (show θ = φ from e) ▸ hφ)
  exact (hneg M ψ).1 (hsound _ _ h2 M hM') (hsound _ _ h1 M hM')

/-- Corollary 3.1(b), classical: `Γ ∪ {¬φ}` is consistent. -/
theorem indep_union_neg_consistent (hsound : Sound Sat) (hcomp : Complete Sat)
    (hneg : NegClause Sat) {Γ : Set Sentence} {φ : Sentence} (hind : Indep Γ φ) :
    Consistent (Γ ∪ {neg φ}) := by
  intro ψ ⟨h1, h2⟩
  obtain ⟨N, hN, hNφ⟩ := (model_split hcomp hneg hind).2
  have hN' : ModelsAll Sat N (Γ ∪ {neg φ}) := fun θ hθ =>
    Or.elim hθ (fun h => hN θ h) (fun e => (show θ = neg φ from e) ▸ hNφ)
  exact (hneg N ψ).1 (hsound _ _ h2 N hN') (hsound _ _ h1 N hN')

/-- Corollary 3.1: an independent sentence is decided, consistently, in either extension. -/
theorem indep_decided_in_extensions (hsound : Sound Sat) (hcomp : Complete Sat)
    (hneg : NegClause Sat) {Γ : Set Sentence} {φ : Sentence} (hind : Indep Γ φ) :
    (Consistent (Γ ∪ {φ}) ∧ Prov (Γ ∪ {φ}) φ) ∧
      (Consistent (Γ ∪ {neg φ}) ∧ Prov (Γ ∪ {neg φ}) (neg φ)) :=
  ⟨⟨indep_union_pos_consistent hsound hcomp hneg hind, assumption (Or.inr rfl)⟩,
   ⟨indep_union_neg_consistent hsound hcomp hneg hind, assumption (Or.inr rfl)⟩⟩
```

Kernel report: `indep_union_pos_consistent`: none; `indep_union_neg_consistent`: propext, Classical.choice, Quot.sound; `indep_decided_in_extensions`: propext, Classical.choice, Quot.sound

Proposition 8.1 Proved. *For every Γ and φ, φ is not independent of $\Gamma \cup \{\varphi\}$.*

```
/-- Independence is relative to `Γ`: the extension `Γ ∪ {φ}` decides `φ`. -/
theorem not_indep_union_self (Γ : Set Sentence) (φ : Sentence) : ¬ Indep (Γ ∪ {φ}) φ :=
  fun h => h.1 (assumption (Or.inr rfl))
```

Kernel report: `not_indep_union_self`: none

Therefore "undecidable in Γ" does not entail "undecidable under every possible extension or reinterpretation." The latter is a different universal statement with a different quantified domain and requires a separate proof. Gödel's theorem does not need to be weakened to make this point. Its own statement already has a formally specified jurisdiction.

9. Syntax is not semantics

A symbol is syntactic. Its denotation or relational role is supplied by an interpretation. Thus the same mark may occur in two structures with different meanings. Formal semantics therefore distinguishes a symbol from the interpretation of the symbol. A proof requires sufficiently fixed interpretation to determine what its formulas assert. This does not imply that the symbol has one immutable meaning under every future use.

Definition 9.1 (Languages, terms, formulas, structures). A language L consists of function symbols and relation symbols, each with an arity. Terms are variables and applications of function symbols to tuples of terms of the right length. Formulas are built from $\bot$, equations $t_1 = t_2$, relation atoms $R(t_1, \dots, t_n)$, the connectives $\neg, \wedge, \vee, \to$ and the quantifiers $\forall, \exists$. An L-structure M on a domain $|M|$ interprets each n-ary function symbol as a map $|M|^n \to |M|$ and each n-ary relation symbol as an n-ary relation on $|M|$. Satisfaction $M \models \varphi[s]$ under an assignment s of elements to variables is defined by the usual recursion.

In the Lean source, variables are numbered and each quantifier binds variable 0, shifting the others (de Bruijn indices); tuples are an inductive family, so that evaluation of terms needs no function extensionality. This encoding changes no statement below.

```
/-- A first-order language: function and relation symbols graded by arity. -/
structure Language where
  Func : Nat → Type
  Rel : Nat → Type

/-- Tuples of length `n`, as an inductive family. -/
inductive Tup (α : Type) : Nat → Type where
  | nil : Tup α 0
  | cons {n : Nat} : α → Tup α n → Tup α (n + 1)

def Tup.map {α β : Type} (f : α → β) : {n : Nat} → Tup α n → Tup β n
  | _, .nil => .nil
  | _, .cons a as => .cons (f a) (Tup.map f as)

mutual
/-- Terms; variables are de Bruijn indices. -/
inductive Term (L : Language) : Type where
  | var : Nat → Term L
  | func {n : Nat} : L.Func n → Terms L n → Term L
/-- Argument lists of length `n`. -/
inductive Terms (L : Language) : Nat → Type where
  | nil : Terms L 0
  | cons {n : Nat} : Term L → Terms L n → Terms L (n + 1)
end

/-- Formulas; `all` and `ex` bind de Bruijn index 0. -/
inductive Formula (L : Language) : Type where
  | falsum : Formula L
  | equal : Term L → Term L → Formula L
  | rel {n : Nat} : L.Rel n → Terms L n → Formula L
  | not : Formula L → Formula L
  | and : Formula L → Formula L → Formula L
  | or : Formula L → Formula L → Formula L
  | imp : Formula L → Formula L → Formula L
  | all : Formula L → Formula L
  | ex : Formula L → Formula L
```

```
/-- An `L`-structure on the domain `α`. -/
structure Structure (L : Language) (α : Type) where
  funMap : {n : Nat} → L.Func n → Tup α n → α
  relMap : {n : Nat} → L.Rel n → Tup α n → Prop

/-- Extend an assignment by a value for de Bruijn index 0. -/
def consAssign {α : Type} (a : α) (v : Nat → α) : Nat → α
  | 0 => a
  | n + 1 => v n

mutual
def Term.eval (M : Structure L α) (v : Nat → α) : Term L → α
  | .var n => v n
  | .func f ts => M.funMap f (Terms.eval M v ts)
def Terms.eval (M : Structure L α) (v : Nat → α) : {n : Nat} → Terms L n → Tup α n
  | _, .nil => .nil
  | _, .cons t ts => .cons (Term.eval M v t) (Terms.eval M v ts)
end

/-- Satisfaction of a formula under an assignment. -/
def Formula.Realize (M : Structure L α) : Formula L → (Nat → α) → Prop
  | .falsum, _ => False
  | .equal t₁ t₂, v => Term.eval M v t₁ = Term.eval M v t₂
  | .rel R ts, v => M.relMap R (Terms.eval M v ts)
  | .not φ, v => ¬ Formula.Realize M φ v
  | .and φ ψ, v => Formula.Realize M φ v ∧ Formula.Realize M ψ v
  | .or φ ψ, v => Formula.Realize M φ v ∨ Formula.Realize M ψ v
  | .imp φ ψ, v => Formula.Realize M φ v → Formula.Realize M ψ v
  | .all φ, v => ∀ a : α, Formula.Realize M φ (consAssign a v)
  | .ex φ, v => ∃ a : α, Formula.Realize M φ (consAssign a v)
```

10. Semantic Stability Theorem

Let $L \subseteq L'$ be languages, let M be an L-structure, and let M' be an L'-structure whose reduct to L is exactly M. That means every symbol already belonging to L retains exactly its previous interpretation.

Definition 10.1 (Language morphism; expansion). A language morphism $\iota : L \to L'$ sends each symbol of L to a symbol of L' of the same arity; an inclusion $L \subseteq L'$ is the case where ι is injective. It translates terms and formulas, $\varphi \mapsto \iota(\varphi)$. An L'-structure M' on the domain of M is an *expansion* of M along ι if $\iota(f)^{M'} = f^{M}$ and $\iota(R)^{M'} = R^{M}$ for every function symbol f and relation symbol R of L; that is, the reduct of M' to L is M.

```
/-- A language morphism, sending each symbol to a symbol of the same arity. An inclusion
`L ⊆ L'` is the case where both maps are injective; injectivity is not used below. -/
structure LHom (L L' : Language) where
  onFunc : {n : Nat} → L.Func n → L'.Func n
  onRel : {n : Nat} → L.Rel n → L'.Rel n

mutual
def LHom.onTerm (ι : LHom L L') : Term L → Term L'
  | .var n => .var n
  | .func f ts => .func (ι.onFunc f) (LHom.onTerms ι ts)
def LHom.onTerms (ι : LHom L L') : {n : Nat} → Terms L n → Terms L' n
  | _, .nil => .nil
```

```
  | _, .cons t ts => .cons (LHom.onTerm ι t) (LHom.onTerms ι ts)
end

def LHom.onFormula (ι : LHom L L') : Formula L → Formula L'
  | .falsum => .falsum
  | .equal t₁ t₂ => .equal (ι.onTerm t₁) (ι.onTerm t₂)
  | .rel R ts => .rel (ι.onRel R) (ι.onTerms ts)
  | .not φ => .not (LHom.onFormula ι φ)
  | .and φ ψ => .and (LHom.onFormula ι φ) (LHom.onFormula ι ψ)
  | .or φ ψ => .or (LHom.onFormula ι φ) (LHom.onFormula ι ψ)
  | .imp φ ψ => .imp (LHom.onFormula ι φ) (LHom.onFormula ι ψ)
  | .all φ => .all (LHom.onFormula ι φ)
  | .ex φ => .ex (LHom.onFormula ι φ)

/-- `M'` is an expansion of `M` along `ι` on the same domain: every old symbol keeps exactly
its old interpretation. (The reduct of `M'` to `L` is `M`.) -/
structure IsExpansion (ι : LHom L L') (M : Structure L α) (M' : Structure L' α) : Prop where
  funMap_eq : ∀ {n : Nat} (f : L.Func n) (xs : Tup α n), M'.funMap (ι.onFunc f) xs = M.funMap f xs
  relMap_iff : ∀ {n : Nat} (R : L.Rel n) (xs : Tup α n), M'.relMap (ι.onRel R) xs ↔ M.relMap R xs
```

Theorem 4 (Adding new vocabulary while preserving old meanings cannot alter old truth) Proved**.** *If M' is an expansion of M along ι, then for every L-formula φ and every assignment s,*

$$M \models \varphi[s] \quad \textit{if and only if} \quad M' \models \iota(\varphi)[s].$$

For an inclusion $L \subseteq L'$, $\iota(\varphi)$ is φ itself read as an L'-formula. Injectivity of ι is not used.

Proof. First, by induction on terms, the value of $\iota(t)$ in M' under s equals the value of t in M under s: variables are unchanged, and at a function symbol the arguments agree by induction while M' interprets $\iota(f)$ as M interprets f. Then proceed by structural induction on φ.

Atomic formulas. All symbols occurring in an L-atomic formula belong to L. By assumption, M' interprets every L-symbol exactly as M does, and the terms have the same values. Therefore the atomic formula has the same truth value in both structures.

Negation. Suppose the result holds for ψ. Then $M \models \neg\psi$ iff $M \not\models \psi$. By the induction hypothesis this is equivalent to $M' \not\models \iota(\psi)$, which is equivalent to $M' \models \neg\iota(\psi)$.

Boolean connectives. The argument is identical componentwise for conjunction, disjunction and implication.

Quantifiers. Suppose the result holds for $\psi(x)$ under every assignment. The underlying domain is unchanged by the expansion. Therefore $M \models \forall x\, \psi(x)$ iff every assignment differing only in x satisfies ψ in M; by induction this occurs iff every such assignment satisfies $\iota(\psi)$ in M', that is, iff $M' \models \forall x\, \iota(\psi)(x)$. Existential quantification follows similarly.

Therefore every old formula retains its truth value. □

```
mutual
theorem eval_onTerm (h : IsExpansion ι M M') (v : Nat → α) :
    (t : Term L) → Term.eval M' v (ι.onTerm t) = Term.eval M v t
  | .var _ => rfl
  | .func f ts => by
      show M'.funMap (ι.onFunc f) (Terms.eval M' v (ι.onTerms ts)) = M.funMap f (Terms.eval M v ts)
      rw [eval_onTerms h v ts]
      exact h.funMap_eq f _
theorem eval_onTerms (h : IsExpansion ι M M') (v : Nat → α) :
    {n : Nat} → (ts : Terms L n) → Terms.eval M' v (ι.onTerms ts) = Terms.eval M v ts
  | _, .nil => rfl
```

```
  | _, .cons t ts => by
      show Tup.cons (Term.eval M' v (ι.onTerm t)) (Terms.eval M' v (ι.onTerms ts)) =
        Tup.cons (Term.eval M v t) (Terms.eval M v ts)
      rw [eval_onTerm h v t, eval_onTerms h v ts]
end

/-- Theorem 4 (semantic stability). -/
theorem realize_onFormula (h : IsExpansion ι M M') :
    (φ : Formula L) → (v : Nat → α) → (Formula.Realize M' (ι.onFormula φ) v ↔ Formula.Realize M φ v)
  | .falsum, _ => Iff.rfl
  | .equal t₁ t₂, v => by
      show Term.eval M' v (ι.onTerm t₁) = Term.eval M' v (ι.onTerm t₂) ↔
        Term.eval M v t₁ = Term.eval M v t₂
      rw [eval_onTerm h v t₁, eval_onTerm h v t₂]
  | .rel R ts, v => by
      show M'.relMap (ι.onRel R) (Terms.eval M' v (ι.onTerms ts)) ↔ M.relMap R (Terms.eval M v ts)
      rw [eval_onTerms h v ts]
      exact h.relMap_iff R _
  | .not φ, v => not_congr (realize_onFormula h φ v)
  | .and φ ψ, v => and_congr (realize_onFormula h φ v) (realize_onFormula h ψ v)
  | .or φ ψ, v => or_congr (realize_onFormula h φ v) (realize_onFormula h ψ v)
  | .imp φ ψ, v => imp_congr (realize_onFormula h φ v) (realize_onFormula h ψ v)
  | .all φ, v => forall_congr' fun a => realize_onFormula h φ (consAssign a v)
  | .ex φ, v => exists_congr fun a => realize_onFormula h φ (consAssign a v)

/-- Theorem 4, consequence: a formula true in `M` stays true, read in any expansion `M'`. -/
theorem realize_onFormula_of_realize {ι : LHom L L'} {M : Structure L α} {M' : Structure L' α}
    (h : IsExpansion ι M M') {φ : Formula L} {v : Nat → α} (hφ : Formula.Realize M φ v) :
    Formula.Realize M' (ι.onFormula φ) v :=
  (realize_onFormula h φ v).2 hφ
```

Kernel report: `eval_onTerm`: none; `realize_onFormula`: none; `realize_onFormula_of_realize`: none

This theorem gives a precise form to the statement: adding a new distinction does not alter an old relation if the interpretation of the old terms is actually preserved. New vocabulary can expose distinctions that were previously inexpressible. It cannot retroactively modify old formulas while simultaneously claiming to have left their meanings fixed. If an old symbol is reinterpreted, that is not a meaning-preserving expansion. It is a changed interpretation and must be identified as such:

Example 10.2 (Reinterpreting an old symbol changes truth) Proved**.** *Let L_P have one unary relation symbol P (and no function symbols), and let $\varphi = \exists x\, P(x)$. On a one-point domain, let M_1 interpret P as true and M_2 interpret the same symbol P as false. Then $M_1 \models \varphi$ and $M_2 \not\models \varphi$.*

Corollary 10.3 Proved**.** *M_2 is not an expansion of M_1 along the identity. Hence the preservation hypothesis of Theorem 4 cannot be dropped.*

Proof. If it were, Theorem 4 would transport the truth of φ from M_1 to M_2. □

```
/-- A language with no function symbols and one relation symbol of each arity. -/
def LP : Language where
  Func _ := Empty
  Rel _ := Unit

/-- `∃x P(x)` with `P` the unary relation symbol of `LP`. -/
def exP : Formula LP := .ex (.rel (n := 1) () (.cons (.var 0) .nil))
```

```
/-- `P` interpreted as true everywhere on a one-point domain. -/
def M₁ : Structure LP Unit where
  funMap f _ := nomatch f
  relMap _ _ := True

/-- The same symbol `P`, reinterpreted as false everywhere. -/
def M₂ : Structure LP Unit where
  funMap f _ := nomatch f
  relMap _ _ := False

theorem reinterpretation_changes_truth (v : Nat → Unit) :
    Formula.Realize M₁ exP v ∧ ¬ Formula.Realize M₂ exP v :=
  ⟨⟨(), trivial⟩, fun ⟨_, h⟩ => h⟩

/-- The identity language morphism. -/
def LHom.id (L : Language) : LHom L L where
  onFunc f := f
  onRel R := R

/-- So `M₂` is not an expansion of `M₁` along the identity: Theorem 4 would otherwise
transport the truth of `exP`. The preservation hypothesis cannot be dropped. -/
theorem reinterpretation_not_expansion : ¬ IsExpansion (LHom.id LP) M₁ M₂ := fun h =>
  (reinterpretation_changes_truth (fun _ => ())).2
    ((realize_onFormula h exP (fun _ => ())).2 (reinterpretation_changes_truth (fun _ => ())).1)
```

Kernel report: `reinterpretation_changes_truth`: none; `reinterpretation_not_expansion`: none

Proposition 10.4 (Reinterpretation as a Deflation instance) Proved from reused. *Let $\iota : L \to L'$ be a language morphism, M an L-structure and M' an L'-structure on the same domain.*

(a) If a relation symbol R of L holds at a tuple in M while $\iota(R)$ fails at that tuple in M', the two readings of the symbol have no common definition: no predicate is equivalent both to R^M and to $\iota(R)^{M'}$.

(b) Hence M' is not an expansion of M along ι.

(c) Example 10.2 is such a case, with ι the identity: the symbol P is shared, and its two readings have no common definition.

Proof. (a) is the Deflation theorem `no_shared_definition` [13], with the new reading as the predicate that excludes the trait "the new reading fails", the old reading as the predicate that admits it, and the given tuple as the witness. (b) An expansion would make R^M a common definition. (c) P^{M_1} holds at the only tuple and P^{M_2} fails there. □

```
/-- No single predicate can be equivalent to both uses. -/
theorem no_shared_definition
    {X : Type} (A B P : X → Prop)
    (hA : ∀ x, A x → ¬ P x)
    (hB : ∃ x, B x ∧ P x) :
    ¬ ∃ S : X → Prop,
      (∀ x, S x ↔ A x) ∧ (∀ x, S x ↔ B x) := ...
```

Kernel report: `no_shared_definition`: none

```
/-- Proposition 10.4(a). If an old relation symbol `R` holds at a tuple in `M` while its image
fails there in `M'`, the two readings of the symbol have no shared definition (Deflation
```

```
`no_shared_definition`, with the new reading as the predicate that excludes the trait). -/
theorem readings_have_no_shared_definition {n : Nat} (R : L.Rel n) (xs : FO.Tup α n)
    (hold : M.relMap R xs) (hnew : ¬ M'.relMap (ι.onRel R) xs) :
    ¬ ∃ S : FO.Tup α n → Prop,
      (∀ ys, S ys ↔ M'.relMap (ι.onRel R) ys) ∧ (∀ ys, S ys ↔ M.relMap R ys) :=
  SemanticDeflation.no_shared_definition (fun ys => M'.relMap (ι.onRel R) ys)
    (fun ys => M.relMap R ys) (fun ys => ¬ M'.relMap (ι.onRel R) ys) (fun _ h hn => hn h)
    ⟨xs, hold, hnew⟩

/-- Proposition 10.4(b). Hence `M'` is not an expansion of `M` along `ι`: a meaning-preserving
expansion would make the old reading a shared definition. -/
theorem not_expansion_of_readings {n : Nat} (R : L.Rel n) (xs : FO.Tup α n)
    (hold : M.relMap R xs) (hnew : ¬ M'.relMap (ι.onRel R) xs) : ¬ FO.IsExpansion ι M M' :=
  fun h => readings_have_no_shared_definition R xs hold hnew
    ⟨fun ys => M.relMap R ys, fun ys => (h.relMap_iff R ys).symm, fun _ => Iff.rfl⟩

/-- Proposition 10.4(c). Example 10.2 is such a case: the symbol `P` is shared, `P` read in `M₁`
holds at the only tuple, and `P` read in `M₂` fails there (compare Deflation
`shared_label_does_not_preserve_meaning`). -/
theorem shared_symbol_does_not_preserve_meaning :
    (FO.LHom.id FO.LP).onRel (n := 1) () = () ∧
    ¬ ∃ S : FO.Tup Unit 1 → Prop,
      (∀ ys, S ys ↔ FO.M₂.relMap (n := 1) () ys) ∧ (∀ ys, S ys ↔ FO.M₁.relMap (n := 1) () ys) :=
  ⟨rfl, readings_have_no_shared_definition (ι := FO.LHom.id FO.LP) (M := FO.M₁) (M' := FO.M₂)
    () (.cons () .nil) trivial id⟩
```

Kernel report: `readings_have_no_shared_definition`: none; `not_expansion_of_readings`: none; `shared_symbol_does_not_preserve_meaning`: none

Theorem 4 is the preservation side. Proposition 10.4 places the failure side in the earlier Deflation theorem, whose finite example uses one public label, "singularity", for two predicates (`shared_label_does_not_preserve_meaning`) Reused. Part I checks the same point for a frozen inscription (`same_input_can_receive_different_interpretations`, `same_label_can_hide_different_meanings`) Reused.

Adding new semantics is therefore not a retroactive refutation of a relation proved under preserved old semantics; a change of the old semantics is a change of structure, and truth is transported only along a proved preservation relation (Theorems 4 and 5).

11. Isomorphism and transport between independently described structures

Definition 11.1 (Isomorphism). An isomorphism $h : M \cong N$ of L-structures is a bijection $h : |M| \to |N|$, given with its inverse, such that $h(f^M(\bar{a})) = f^N(h\bar{a})$ for every function symbol f, and $R^N(h\bar{a}) \iff R^M(\bar{a})$ for every relation symbol R.

Theorem 5 (Isomorphic structures preserve first-order relations) Proved**.** *If $h : M \cong N$, then for every L-formula φ and assignment s,*

$$M \models \varphi[s] \quad \textit{if and only if} \quad N \models \varphi[h \circ s].$$

Proof. By induction on terms and formulas, proving the stronger statement for any assignments s, w with $w(n) = h(s(n))$ for all n. For terms, the value under w in N is h of the value under s in M, because h commutes with the function symbols. The atomic case follows because an isomorphism preserves the interpretations of constants, functions and relations; for equations, $h(a) = h(b)$ iff $a = b$ because h is injective. The Boolean cases follow immediately from induction. The quantifier cases follow because h is bijective, so ranging over the domain of M corresponds exactly to ranging over the domain of N: an element b of N is $h(h^{-1}(b))$. □

```
/-- An isomorphism of `L`-structures: a bijection (given with its inverse) commuting with every
function symbol and preserving every relation symbol in both directions. -/
structure Iso (M : Structure L α) (N : Structure L β) where
  toFun : α → β
  invFun : β → α
  left_inv : ∀ a, invFun (toFun a) = a
  right_inv : ∀ b, toFun (invFun b) = b
  map_fun : ∀ {n : Nat} (f : L.Func n) (xs : Tup α n), toFun (M.funMap f xs) = N.funMap f (xs.map toFun)
  map_rel : ∀ {n : Nat} (R : L.Rel n) (xs : Tup α n), N.relMap R (xs.map toFun) ↔ M.relMap R xs

mutual
theorem Iso.eval_term (h : Iso M N) {v : Nat → α} {w : Nat → β}
    (hvw : ∀ n, w n = h.toFun (v n)) : (t : Term L) → Term.eval N w t = h.toFun (Term.eval M v t)
  | .var n => hvw n
  | .func f ts => by
      show N.funMap f (Terms.eval N w ts) = h.toFun (M.funMap f (Terms.eval M v ts))
      rw [Iso.eval_terms h hvw ts, h.map_fun]
theorem Iso.eval_terms (h : Iso M N) {v : Nat → α} {w : Nat → β}
    (hvw : ∀ n, w n = h.toFun (v n)) :
    {n : Nat} → (ts : Terms L n) → Terms.eval N w ts = (Terms.eval M v ts).map h.toFun
  | _, .nil => rfl
  | _, .cons t ts => by
      show Tup.cons (Term.eval N w t) (Terms.eval N w ts) =
        Tup.cons (h.toFun (Term.eval M v t)) ((Terms.eval M v ts).map h.toFun)
      rw [Iso.eval_term h hvw t, Iso.eval_terms h hvw ts]
end

theorem Iso.cons_pointwise (h : Iso M N) {v : Nat → α} {w : Nat → β}
    (hvw : ∀ n, w n = h.toFun (v n)) {a : α} {b : β} (hab : b = h.toFun a) :
    ∀ n, consAssign b w n = h.toFun (consAssign a v n)
  | 0 => hab
  | n + 1 => hvw n

theorem Iso.toFun_inj (h : Iso M N) {a a' : α} (e : h.toFun a = h.toFun a') : a = a' :=
  (h.left_inv a).symm.trans ((congrArg h.invFun e).trans (h.left_inv a'))

/-- Theorem 5, assignment-pointwise form. -/
theorem Iso.realize (h : Iso M N) : (φ : Formula L) → {v : Nat → α} → {w : Nat → β} →
    (∀ n, w n = h.toFun (v n)) → (Formula.Realize N φ w ↔ Formula.Realize M φ v)
  | .falsum, _, _, _ => Iff.rfl
  | .equal t₁ t₂, v, w, hvw => by
      show Term.eval N w t₁ = Term.eval N w t₂ ↔ Term.eval M v t₁ = Term.eval M v t₂
      rw [h.eval_term hvw t₁, h.eval_term hvw t₂]
      exact ⟨h.toFun_inj, congrArg h.toFun⟩
  | .rel R ts, v, w, hvw => by
      show N.relMap R (Terms.eval N w ts) ↔ M.relMap R (Terms.eval M v ts)
      rw [h.eval_terms hvw ts]
      exact h.map_rel R _
  | .not φ, _, _, hvw => not_congr (h.realize φ hvw)
  | .and φ ψ, _, _, hvw => and_congr (h.realize φ hvw) (h.realize ψ hvw)
  | .or φ ψ, _, _, hvw => or_congr (h.realize φ hvw) (h.realize ψ hvw)
  | .imp φ ψ, _, _, hvw => imp_congr (h.realize φ hvw) (h.realize ψ hvw)
  | .all φ, _, _, hvw =>
      ⟨fun hN a => (h.realize φ (h.cons_pointwise hvw rfl)).1 (hN (h.toFun a)),
       fun hM b => (h.realize φ (h.cons_pointwise hvw (h.right_inv b).symm)).2 (hM (h.invFun b))⟩
  | .ex φ, _, _, hvw =>
      ⟨fun ⟨b, hb⟩ => ⟨h.invFun b, (h.realize φ (h.cons_pointwise hvw (h.right_inv b).symm)).1 hb⟩,
       fun ⟨a, ha⟩ => ⟨h.toFun a, (h.realize φ (h.cons_pointwise hvw rfl)).2 ha⟩⟩
```

```
/-- Theorem 5: `M` satisfies `φ` under `s` iff `N` satisfies `φ` under `h ∘ s`. -/
theorem Iso.realize_comp (h : Iso M N) (φ : Formula L) (s : Nat → α) :
    Formula.Realize M φ s ↔ Formula.Realize N φ (fun n => h.toFun (s n)) :=
  (h.realize φ (fun _ => rfl)).symm
```

Kernel report: `eval_term`: none; `cons_pointwise`: none; `toFun_inj`: none; `realize`: none; `realize_comp`: none

Proposition 11.2 (Both halves of bijectivity are needed) Proved. *Let L_0 be the empty language and $\theta = \exists x\, \exists y\, \neg(x = y)$. Then θ holds on a two-element domain and fails on a one-element domain. The map from the two-element domain onto the one-element domain is surjective and commutes with every symbol of L_0; the map from the one-element domain into the two-element domain is injective and commutes with every symbol of L_0. Neither preserves the truth of θ.*

```
/-- A map of structures commuting with function symbols and preserving relation symbols in both
directions, with no bijectivity assumed. -/
structure Hom (M : Structure L α) (N : Structure L β) where
  toFun : α → β
  map_fun : ∀ {n : Nat} (f : L.Func n) (xs : Tup α n), toFun (M.funMap f xs) = N.funMap f (xs.map toFun)
  map_rel : ∀ {n : Nat} (R : L.Rel n) (xs : Tup α n), N.relMap R (xs.map toFun) ↔ M.relMap R xs

/-- The empty language. -/
def L0 : Language where
  Func _ := Empty
  Rel _ := Empty

/-- The structure of the empty language on a domain. -/
def bare (α : Type) : Structure L0 α where
  funMap f _ := nomatch f
  relMap R _ := nomatch R

/-- `∃x ∃y ¬(x = y)`: there are two distinct elements. -/
def twoDistinct : Formula L0 := .ex (.ex (.not (.equal (.var 0) (.var 1))))

theorem twoDistinct_bool (v : Nat → Bool) : Formula.Realize (bare Bool) twoDistinct v :=
  ⟨true, false, fun h => Bool.noConfusion h⟩

theorem twoDistinct_unit (v : Nat → Unit) : ¬ Formula.Realize (bare Unit) twoDistinct v :=
  fun ⟨_, _, h⟩ => h rfl

/-- A surjective, non-injective map preserving all symbols: truth is not preserved. -/
def collapse : Hom (bare Bool) (bare Unit) where
  toFun _ := ()
  map_fun f _ := nomatch f
  map_rel R _ := nomatch R

theorem injectivity_needed :
    (∀ b : Unit, ∃ a : Bool, collapse.toFun a = b) ∧
      Formula.Realize (bare Bool) twoDistinct (fun _ => true) ∧
      ¬ Formula.Realize (bare Unit) twoDistinct (fun _ => collapse.toFun true) :=
  ⟨fun _ => ⟨true, rfl⟩, twoDistinct_bool _, twoDistinct_unit _⟩

/-- An injective, non-surjective map preserving all symbols: truth is not preserved. -/
def embedPoint : Hom (bare Unit) (bare Bool) where
  toFun _ := true
  map_fun f _ := nomatch f
```

```
  map_rel R _ := nomatch R

theorem surjectivity_needed :
    (∀ a a' : Unit, embedPoint.toFun a = embedPoint.toFun a' → a = a') ∧
      ¬ Formula.Realize (bare Unit) twoDistinct (fun _ => ()) ∧
      Formula.Realize (bare Bool) twoDistinct (fun _ => embedPoint.toFun ()) :=
  ⟨fun _ _ _ => rfl, twoDistinct_unit _, twoDistinct_bool _⟩
```

Kernel report: `twoDistinct_bool`: none; `twoDistinct_unit`: none; `injectivity_needed`: none; `surjectivity_needed`: none

Proposition 11.3 (Transport is a predicate-preserving translation) Proved from reused. *(a) An isomorphism $h : M \cong N$ induces, for every formula φ, the translation of assignments $s \mapsto h \circ s$, which preserves the predicate "satisfies φ" in the sense of Part I [14]. (b) Two isomorphisms $M \cong N \cong K$ compose to such a translation from M to K.*

Proof. (a) is Theorem 5. (b) is Part I's `predicate_preservation_composes` applied to (a) twice. □

```
/-- Proposition 11.3(a). An isomorphism is a predicate-preserving translation of assignments in
the sense of Part I (`PreservesPredicate`), for every formula (Theorem 5). -/
theorem iso_preserves_predicate {L : FO.Language} {α β : Type} {M : FO.Structure L α}
    {N : FO.Structure L β} (h : FO.Iso M N) (φ : FO.Formula L) :
    LambdaSpatiotemporal.PreservesPredicate (fun s : Nat → α => fun n => h.toFun (s n))
      (FO.Formula.Realize M φ) (FO.Formula.Realize N φ) :=
  fun s => h.realize_comp φ s

/-- Proposition 11.3(b). Transport composes along two isomorphisms (Part I
`predicate_preservation_composes`). -/
theorem iso_transport_composes {L : FO.Language} {α β γ : Type} {M : FO.Structure L α}
    {N : FO.Structure L β} {K : FO.Structure L γ} (h₁ : FO.Iso M N) (h₂ : FO.Iso N K)
    (φ : FO.Formula L) :
    LambdaSpatiotemporal.PreservesPredicate
      (fun s : Nat → α => fun n => h₂.toFun (h₁.toFun (s n)))
      (FO.Formula.Realize M φ) (FO.Formula.Realize K φ) :=
  LambdaSpatiotemporal.predicate_preservation_composes
    (fun s : Nat → α => fun n => h₁.toFun (s n)) (fun t : Nat → β => fun n => h₂.toFun (t n))
    (FO.Formula.Realize M φ) (FO.Formula.Realize N φ) (FO.Formula.Realize K φ)
    (iso_preserves_predicate h₁ φ) (iso_preserves_predicate h₂ φ)
```

Kernel report: `iso_preserves_predicate`: none; `iso_transport_composes`: none

Part I also checks that translations need not commute (`translations_need_not_commute`) Reused: preservation is a property of the given maps, established by theorem, not a property of translation as such. R4 checks the corresponding statement for warrants, `warranted_translation_needs_a_bridge`, and the Dimensional Ladder checks that a commuting bridge transports identity of evaluations, `commuting_bridge_transports_identity` Reused.

Thus the relation between two independently presented structures does not have to be taken as a primitive given. It can itself be proved. Once proved, relevant consequences can be transported. What is asserted is the isomorphism h, and what is transported is the truth of L-formulas; identity of the structures is neither required nor asserted.

12. Self-application of an equal proof standard

Fix a language L; explicit assumptions Γ; a consequence relation $\vdash$ satisfying the structural rules of Section 1; and fixed interpretations for the terms used in the claim, which in the deductive

setting are fixed by the sentences and the map $\neg$. A conclusion φ is accepted exactly when an adequate derivation establishes $\Gamma \vdash \varphi$. A claimed derivation of φ uses premises Γ_0. An objection derives a sentence χ from premises Δ by means of a relation $\vdash'$, the inferences the objector uses.

Theorem 6 (Symmetric Adjudication Theorem) Proved**.** *Consider the conditions*

(1) $\Gamma_0 \subseteq \Gamma$*: every premise used is admitted;*

(2) $\Gamma_0 \vdash \varphi$*: the inference is valid;*

(3) Γ *is consistent;*

(4) every member of Δ *is derivable from* Γ*: the objection introduces no new premise;*

(5) φ *and* χ *are contradictory: under the fixed interpretation, the objection's conclusion is the negation of the claim, or the claim is the negation of the objection's conclusion;*

(6) every inference of $\vdash'$ *is an inference of* $\vdash$*: the consequence relation is unchanged.*

Then:

(a) conditions (1)–(6) and $\Delta \vdash' \chi$ *cannot all hold;*

(b) if $\Delta \vdash' \chi$*, then at least one of (1)–(6) fails: an alleged premise was not admitted, an inference is invalid,* Γ *is inconsistent, some* $\delta \in \Delta$ *with* $\Gamma \nvdash \delta$ *is a genuinely new premise, the objection concerns a reinterpreted claim, or the consequence relation has changed;*

(c) the conditions are symmetric in claim and objection, so the same theorem, with roles exchanged, governs any challenge to an established objection.

Proof. (a) By (1), (2) and monotonicity, $\Gamma \vdash \varphi$. By (6), $\Delta \vdash \chi$; by (4) and cut, $\Gamma \vdash \chi$. By (5) and Lemma 4.3 this contradicts (3).

(b) follows from (a) by classical case analysis on the six conditions.

(c) Definition 4.2 is symmetric, so (5) holds for (φ, χ) exactly when it holds for (χ, φ); apply (a) with the roles of the two derivations exchanged. □

```
/-- Theorem 6, constructive core. A claimed derivation of `φ` from premises `Γ₀` and an objection
deriving `χ` from premises `Δ` under a relation `Prov'` cannot satisfy all six conditions at once:
(1) the premises of the claimed derivation are admitted, `Γ₀ ⊆ Γ`;
(2) the claimed inference is valid, `Γ₀ ⊢ φ`;
(3) the admitted premises are consistent;
(4) every premise of the objection is derivable from `Γ`;
(5) under the fixed reading, the objection's conclusion contradicts `φ`;
(6) every inference available to the objection is available to `⊢`. -/
theorem symmetric_adjudication_core {Γ Γ₀ Δ : Set Sentence} {φ χ : Sentence}
    {Prov' : Set Sentence → Sentence → Prop}
    (hadm : Γ₀ ⊆ Γ) (hinf : Prov Γ₀ φ) (hcons : Consistent Γ)
    (hold : ∀ δ ∈ Δ, Prov Γ δ) (hint : Contradictory φ χ)
    (hrel : ∀ θ ψ, Prov' θ ψ → Prov θ ψ) (hobj : Prov' Δ χ) : False :=
  hcons.not_both hint (monotonicity hadm hinf, cut hold (hrel Δ χ hobj))

/-- Theorem 6, disjunctive form (classical): a successful objection falsifies at least one of the
six conditions. -/
theorem symmetric_adjudication {Γ Γ₀ Δ : Set Sentence} {φ χ : Sentence}
    {Prov' : Set Sentence → Sentence → Prop} (hobj : Prov' Δ χ) :
    ¬ Γ₀ ⊆ Γ ∨ ¬ Prov Γ₀ φ ∨ ¬ Consistent Γ ∨ (∃ δ ∈ Δ, ¬ Prov Γ δ) ∨
      ¬ Contradictory φ χ ∨ (∃ θ ψ, Prov' θ ψ ∧ ¬ Prov θ ψ) := by
  by_cases h1 : Γ₀ ⊆ Γ
  · by_cases h2 : Prov Γ₀ φ
```

```
    · by_cases h3 : Consistent Γ
      · by_cases h4 : ∀ δ ∈ Δ, Prov Γ δ
        · by_cases h5 : Contradictory φ χ
          · by_cases h6 : ∀ θ ψ, Prov' θ ψ → Prov θ ψ
            · exact (symmetric_adjudication_core h1 h2 h3 h4 h5 h6 hobj).elim
            · push Not at h6
              exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr h6))))
          · exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inl h5))))
        · push Not at h4
          exact Or.inr (Or.inr (Or.inr (Or.inl h4)))
      · exact Or.inr (Or.inr (Or.inl h3))
    · exact Or.inr (Or.inl h2)
  · exact Or.inl h1

/-- Self-application: the same theorem governs a challenge to an established objection. -/
theorem symmetric_adjudication_core_swap {Γ Γ₀ Δ : Set Sentence} {φ χ : Sentence}
    {Prov' : Set Sentence → Sentence → Prop}
    (hadm : Γ₀ ⊆ Γ) (hinf : Prov Γ₀ χ) (hcons : Consistent Γ)
    (hold : ∀ δ ∈ Δ, Prov Γ δ) (hint : Contradictory φ χ)
    (hrel : ∀ θ ψ, Prov' θ ψ → Prov θ ψ) (hobj : Prov' Δ φ) : False :=
  symmetric_adjudication_core hadm hinf hcons hold hint.symm hrel hobj
```

Kernel report: `symmetric_adjudication_core`: none; `symmetric_adjudication`: propext, Classical.choice, Quot.sound; `symmetric_adjudication_core_swap`: none

If the objection uses no additional premises and the same consequence relation, then a successful objection would establish $\Gamma \vdash \neg\varphi$, which together with $\Gamma \vdash \varphi$ contradicts consistency. If the objection uses additional premises Δ, Theorem 2 proves that at least one of them is not already derivable from Γ, unless the original assumptions are inconsistent. If an interpretation is changed, the argument concerns a different interpreted structure unless an explicit preservation theorem transports the result (Theorems 4 and 5, Example 10.2). If the inference rules are changed, the argument is no longer made under the original consequence relation. A countermodel, where soundness holds, is one way to establish that an inference is invalid (Lemma 7.3).

Mere non-acceptance supplies no additional logical possibility: it is not a derivation of $\neg\varphi$ (Proposition 5.1), and making it one would be a change of the consequence relation (Proposition 5.3).

Proposition 12.1 (No refutation has priority without a new premise) Proved from reused. *Let $\Gamma \vdash \varphi$ with Γ consistent. Call a premise set Δ* prior *if $\Gamma \cup \Delta \vdash \neg\varphi$, and* complement-using *if some $\delta \in \Delta$ has $\Gamma \nvdash \delta$. Then no Δ is prior without being complement-using.*

Proof. Theorem 2 shows that every prior Δ is complement-using. That is the hypothesis of Part I's priority schema `complement_requirement_blocks_absolute_priority` (K14) [14], whose conclusion is the statement. □

```
/-- Proposition 12.1. Theorem 2 discharges the hypothesis of Part I's priority schema
(`complement_requirement_blocks_absolute_priority`): for a sentence established from a consistent
`Γ`, no premise set derives its negation without a premise new to `Γ`. -/
theorem no_complement_free_refutation {Sentence : Type} [Consequence Sentence]
    {Γ : Set Sentence} {φ : Sentence} (hpos : Prov Γ φ) (hcons : Consistent Γ) :
    ¬ ∃ Δ : Set Sentence, LambdaSpatiotemporal.AbsolutePriority
      (fun Δ => Prov (Γ ∪ Δ) (neg φ)) (fun Δ => ∃ δ ∈ Δ, ¬ Prov Γ δ) Δ :=
  LambdaSpatiotemporal.complement_requirement_blocks_absolute_priority _ _
    fun _ h => negative_derivation_requires_new_premise hpos hcons h
```

Kernel report: `no_complement_free_refutation`: propext, Classical.choice, Quot.sound

Part III restates the schema as `LambdaIII.complement_requirement_blocks_absolute_priority` and proves a cyclic form, `no_first_position_cyclic`; R4 adds `different_origin_rankings_need_no_absolute_comparator` Reused. These are structural antecedents of the remark that the adjudicating position supplies no inference rule. Part III also carries the two adjudication results of *Equal Standard* [21]: under a fair standard, equal declared criteria force equal verdicts (`equal_criteria_forces_equal_verdict`), and two cases with equal criteria are both accepted, both rejected, or expose an undeclared criterion (`equal_standard_trichotomy`) Reused.

13. Why this theorem is not self-sealing

The theorem explicitly permits its own defeat. A critic may show that a premise is false; that a premise was never actually admitted; that a derivation step is invalid; that the supposed consequence relation lacks cut or another stated property; that the interpretation was incorrectly specified; that an alleged isomorphism fails; that the theory is inconsistent; or that the theorem has exceeded its declared scope. Any one of these is a genuine mathematical objection: each negates a hypothesis of Theorem 6 or of the result it is applied to.

What is excluded is only the inference: "I have supplied no defect and no counter-derivation, but I remain unconvinced; therefore the proposition has been refuted." That does not have the logical form of a refutation.

The direct formal basis of this section is Theorem 6(c), `symmetric_adjudication_core_swap` Proved: the six conditions are symmetric in claim and objection, so the same theorem, with roles exchanged, governs a challenge to an established objection, and a challenge to an application of Theorem 6 itself. Each defect in the list above negates one of conditions (1)–(6) of the result it is applied to, so the list is the theorem's own criterion of defeat. R4 checks the same self-application for its encoding of assessments: a challenge is an instance of the assessment relation (`challenge_is_an_instance`), every consideration has an applied rule (`consideration_has_an_applied_rule`), and a receipt that a rule produced a verdict does not certify the rule's soundness (`receipt_does_not_certify_rule_soundness`) Reused. A self-sealing procedure, in the sense of the Self-Sealing Openness theorem of [19], has no terminal route to the universal state; Theorem 6 states its terminal routes.

14. The formal analogue of the "privileged viewpoint" error

Suppose a proof establishes $\Gamma \vdash \varphi$. That does not by itself establish "φ under every possible extension, interpretation, language, and future distinction." The second statement has a larger quantifier domain. To prove it, that domain must itself be specified and covered by an appropriate theorem. Silently moving from all models or interpretations covered by this proof to all possible meanings whatsoever is a quantifier-scope change. It is not licensed by the original derivation.

Equally, however, the possibility of another interpretation does not refute what was proved under the original one. Example 10.2 exhibits both points at once: φ holds under one interpretation of P and fails under another, so truth under the first does not extend to all interpretations, and failure under the second does not touch the first. Theorem 4 states exactly which changes of language leave truth fixed. Both errors are prevented by keeping the scope explicit.

Proposition 14.1 (Truth under one interpretation is a local warrant) Proved from reused**.** *Index the two structures of Example 10.2 by Part I's two frames,* local *for* M_1 *and* other *for* M_2*. Frame by frame, the truth of* $\exists x\, P(x)$ *is Part I's sample warrant. Hence it holds in the local frame and is not a universal warrant over both frames.*

```
/-- The two structures of Example 10.2, indexed by Part I's two frames. -/
def frameStructure : LambdaSpatiotemporal.TwoFrame → FO.Structure FO.LP Unit
  | .local => FO.M₁
  | .other => FO.M₂

/-- Frame by frame, the truth of `∃x P(x)` is Part I's sample warrant. -/
theorem frame_truth_iff_sample_warrant (v : Nat → Unit) :
    (f : LambdaSpatiotemporal.TwoFrame) →
      (FO.Formula.Realize (frameStructure f) FO.exP v ↔ LambdaSpatiotemporal.SampleWarrant f)
  | .local => ⟨fun _ => trivial, fun _ => (FO.reinterpretation_changes_truth v).1⟩
  | .other => ⟨fun h => (FO.reinterpretation_changes_truth v).2 h, fun h => False.elim h⟩

/-- Proposition 14.1. Truth of `∃x P(x)` holds in the local frame and is not a universal warrant
over both frames (Part I `local_warrant_not_universal_in_general`, transported along
`frame_truth_iff_sample_warrant`). -/
theorem truth_is_local_warrant (v : Nat → Unit) :
    FO.Formula.Realize (frameStructure .local) FO.exP v ∧
    ¬ LambdaSpatiotemporal.UniversalWarrant
        (fun f => FO.Formula.Realize (frameStructure f) FO.exP v) :=
  ⟨((frame_truth_iff_sample_warrant v .local).2
      LambdaSpatiotemporal.local_warrant_not_universal_in_general.1,
   fun h => LambdaSpatiotemporal.local_warrant_not_universal_in_general.2
      fun f => (frame_truth_iff_sample_warrant v f).1 (h f))⟩
```

Kernel report: `frame_truth_iff_sample_warrant`: none; `truth_is_local_warrant`: none

The conclusion is Part I's `local_warrant_not_universal_in_general` Reused, transported along the frame-by-frame equivalence. Part I also checks that a predicate named "absolute" has no elimination rule by logic alone (`absolute_label_does_not_self_license`) Reused.

15. Class-level closure by an empty solution set

Many familiar theorems have exactly the form under discussion. Let C be a defined class and let $R(x)$ be a fixed relation. Define its solution set $Z_R = \{x \in C : R(x)\}$.

Proposition 15.1 Proved**.** *$\forall x \in C,\ \neg R(x)$ holds if and only if $Z_R = \varnothing$.*

```
/-- Null closure, membership form. -/
theorem null_closure_mem {α : Type*} (C : Set α) (R : α → Prop) :
    (∀ x ∈ C, ¬ R x) ↔ ∀ x, x ∉ {x ∈ C | R x} :=
  ⟨fun h x hx => h x hx.1 hx.2, fun h x hC hR => h x ⟨hC, hR⟩⟩

/-- Null closure, set-equality form. -/
theorem null_closure {α : Type*} (C : Set α) (R : α → Prop) :
    (∀ x ∈ C, ¬ R x) ↔ {x ∈ C | R x} = ∅ :=
  (null_closure_mem C R).trans Set.eq_empty_iff_forall_notMem.symm
```

Kernel report: `null_closure_mem`: none; `null_closure`: propext, Quot.sound

This is a class-level closure. It does not require enumeration of every member of C.

16. Polygon example

For an inscribed regular n-gon in a unit circle, $P_n = 2n\sin(\pi/n)$.

```
/-- Perimeter of the regular `n`-gon inscribed in the unit circle. -/
noncomputable def P (n : ℕ) : ℝ := 2 * n * Real.sin (π / n)
```

Theorem 16.1 Mathlib. *For every integer $n \geq 1$, $P_n < 2\pi$. Consequently, for every finite integer $n \geq 3$, $P_n \neq 2\pi$, and $\{n \geq 3 : P_n = 2\pi\} = \varnothing$.*

Proof. For $n \geq 1$ we have $0 < \pi/n$, and the standard inequality $\sin x < x$ for positive x gives $2n \sin(\pi/n) < 2n(\pi/n) = 2\pi$. □

The inequality uses only $0 < \pi/n$, so it holds for every $n \geq 1$; the range $n \geq 3$ is where P_n is the perimeter of an inscribed regular polygon. The inequality $\sin x < x$ is Mathlib's `Real.sin_lt`.

```
/-- `P n < 2π` for every `n ≥ 1`. -/
theorem P_lt_two_pi {n : ℕ} (hn : 1 ≤ n) : P n < 2 * π := by
  have hn' : (0 : ℝ) < n := by exact_mod_cast hn
  have hx : 0 < π / n := div_pos pi_pos hn'
  calc P n = 2 * n * Real.sin (π / n) := rfl
    _ < 2 * n * (π / n) := mul_lt_mul_of_pos_left (Real.sin_lt hx) (by positivity)
    _ = 2 * π := by field_simp

/-- The polygon class `n ≥ 3`. -/
theorem polygon_perimeter_lt {n : ℕ} (hn : 3 ≤ n) : P n < 2 * π := P_lt_two_pi (by omega)

/-- No finite polygon in the class closes the equality. -/
theorem polygon_equality_set_empty : {n : ℕ | 3 ≤ n ∧ P n = 2 * π} = ∅ :=
  Set.eq_empty_of_forall_notMem fun _ h => (polygon_perimeter_lt h.1).ne h.2
```

Kernel report: `P_lt_two_pi`: propext, Classical.choice, Quot.sound; `polygon_perimeter_lt`: propext, Classical.choice, Quot.sound; `polygon_equality_set_empty`: propext, Classical.choice, Quot.sound

Theorem 16.2 Mathlib. *For every $n \geq 1$, $2\pi - \dfrac{\pi^3}{3n^2} < P_n < 2\pi$. Hence $|2\pi - P_n| < \dfrac{\pi^3}{3n^2}$, and $P_n \to 2\pi$.*

Proof. For $x > 0$, $\sin x > x - x^3/6$. With $x = \pi/n$, $P_n > 2n\big(\pi/n - \pi^3/(6n^3)\big) = 2\pi - \pi^3/(3n^2)$. The upper bound is Theorem 16.1. Since $\pi^3/(3n^2) \leq (\pi^3/3)/n \to 0$, the two bounds squeeze P_n to 2π. □

The lower inequality for sin is Mathlib's `Real.sin_gt_sub_cube`, and $c/n \to 0$ is `tendsto_const_div_atTop_nhds_zero_nat`.

```
/-- Lower bound: `2π - π³/(3n²) < P n` for every `n ≥ 1`. -/
theorem two_pi_sub_lt_P {n : ℕ} (hn : 1 ≤ n) : 2 * π - π ^ 3 / (3 * n ^ 2) < P n := by
  have hn' : (0 : ℝ) < n := by exact_mod_cast hn
  have hx : 0 < π / n := div_pos pi_pos hn'
  have key : 2 * (n : ℝ) * (π / n - (π / n) ^ 3 / 6) = 2 * π - π ^ 3 / (3 * n ^ 2) := by
    field_simp
    ring
  calc 2 * π - π ^ 3 / (3 * n ^ 2) = 2 * (n : ℝ) * (π / n - (π / n) ^ 3 / 6) := key.symm
    _ < 2 * n * Real.sin (π / n) :=
        mul_lt_mul_of_pos_left (Real.sin_gt_sub_cube hx) (by positivity)

/-- Explicit rate: `|2π - P n| < π³/(3n²)` for every `n ≥ 1`. -/
theorem abs_two_pi_sub_P_lt {n : ℕ} (hn : 1 ≤ n) : |2 * π - P n| < π ^ 3 / (3 * n ^ 2) := by
  rw [abs_of_pos (by linarith [P_lt_two_pi hn])]
  linarith [two_pi_sub_lt_P hn]
```

```
/-- `P n → 2π`. -/
theorem P_tendsto : Tendsto P atTop (nhds (2 * π)) := by
  have hlow : Tendsto (fun n : ℕ => 2 * π - (π ^ 3 / 3) / (n : ℝ)) atTop (nhds (2 * π)) := by
    have hc : Tendsto (fun _ : ℕ => 2 * π) atTop (nhds (2 * π)) := tendsto_const_nhds
    simpa using hc.sub (tendsto_const_div_atTop_nhds_zero_nat (π ^ 3 / 3))
  refine tendsto_of_tendsto_of_tendsto_of_le_of_le' hlow tendsto_const_nhds ?_ ?_
  · filter_upwards [eventually_ge_atTop 1] with n hn
    have hn' : (1 : ℝ) ≤ n := by exact_mod_cast hn
    have h := two_pi_sub_lt_P hn
    have hle : π ^ 3 / (3 * (n : ℝ) ^ 2) ≤ (π ^ 3 / 3) / n := by
      rw [div_div]
      exact div_le_div_of_nonneg_left (by positivity) (by linarith) (by nlinarith)
    linarith
  · filter_upwards [eventually_ge_atTop 1] with n hn
    exact (P_lt_two_pi hn).le

/-- Approach without attainment. -/
theorem P_approaches_not_attained :
    Tendsto P atTop (nhds (2 * π)) ∧ ∀ n : ℕ, 1 ≤ n → P n ≠ 2 * π :=
  ⟨P_tendsto, fun _ hn => (P_lt_two_pi hn).ne⟩
```

Kernel report: `two_pi_sub_lt_P`: propext, Classical.choice, Quot.sound; `abs_two_pi_sub_P_lt`: propext, Classical.choice, Quot.sound; `P_tendsto`: propext, Classical.choice, Quot.sound; `P_approaches_not_attained`: propext, Classical.choice, Quot.sound

The polygons converge to the circle in perimeter, $P_n \to 2\pi$, but no finite polygon in the specified class closes the equality. Approaching does not mean equality at some finite stage. This is proved class-wide by a relation, not by enumerating polygons.

17. Exact proportion as an integer relation

A clean rational proportion between a quantity x and 1 exists exactly when there are integers a, b, with $a \neq 0$, satisfying $ax = b$.

Proposition 17.1 Proved. *For real x: $x \in \mathbb{Q}$ if and only if there are $a, b \in \mathbb{Z}$ with $a \neq 0$ and $ax - b = 0$.*

Proof. If $x = q$ with $q \in \mathbb{Q}$ in lowest terms with denominator $d > 0$ and numerator m, take $a = d$, $b = m$. Conversely, $ax - b = 0$ with $a \neq 0$ gives $x = b/a \in \mathbb{Q}$. □

Theorem 17.2 Mathlib. *For all $a, b \in \mathbb{Z}$, if $a \neq 0$ then $a\pi - b \neq 0$. Equivalently, $\{(a, b) \in \mathbb{Z}^2 : a \neq 0 \wedge a\pi = b\} = \varnothing$.*

Proof. By Proposition 17.1, a solution would make π rational, contradicting the irrationality of π (Mathlib's `irrational_pi`). □

```
/-- `x` is rational iff `a * x - b = 0` for integers `a ≠ 0` and `b`. -/
theorem rational_iff_int_relation (x : ℝ) :
    x ∈ Set.range ((↑) : ℚ → ℝ) ↔ ∃ a b : ℤ, a ≠ 0 ∧ (a : ℝ) * x - b = 0 := by
  constructor
  · rintro ⟨q, rfl⟩
    refine ⟨q.den, q.num, by exact_mod_cast q.den_nz, ?_⟩
    have hden : (q.den : ℝ) ≠ 0 := by exact_mod_cast q.den_nz
    rw [Rat.cast_def]
    push_cast
```

```
    field_simp
    ring
  · rintro ⟨a, b, ha, h⟩
    refine ⟨(b : ℚ) / a, ?_⟩
    have ha' : (a : ℝ) ≠ 0 := by exact_mod_cast ha
    push_cast
    field_simp
    linarith

/-- No nontrivial integer relation closes on `π` (from Mathlib's `irrational_pi`). -/
theorem pi_int_relation_null : ∀ a b : ℤ, a ≠ 0 → (a : ℝ) * π - b ≠ 0 :=
  fun a b ha h => irrational_pi ((rational_iff_int_relation π).2 ⟨a, b, ha, h⟩)

/-- The integer-relation solution set for `π` is empty. -/
theorem pi_int_relation_set_empty :
    {ab : ℤ × ℤ | ab.1 ≠ 0 ∧ (ab.1 : ℝ) * π = ab.2} = ∅ :=
  Set.eq_empty_of_forall_notMem fun ab h =>
    pi_int_relation_null ab.1 ab.2 h.1 (sub_eq_zero.2 h.2)
```

Kernel report: `rational_iff_int_relation`: propext, Classical.choice, Quot.sound; `pi_int_relation_null`: propext, Classical.choice, Quot.sound; `pi_int_relation_set_empty`: propext, Classical.choice, Quot.sound

R4 checks a related representation statement from the same Mathlib theorem: $\pi \neq a/b^n$ for every integer a and all natural numbers b and n (`pi_no_finite_base_fraction`) Reused.

The claim concerns an exact integer relation, not the quality of approximations to π. No finite search over integer ratios establishes this. A proof of irrationality closes the entire defined class at once.

18. Transcendence as stronger class-level closure

A number x is algebraic precisely when there exists a nonzero polynomial $p(X) \in \mathbb{Z}[X]$ such that $p(x) = 0$. This is Mathlib's definition of `IsAlgebraic ℤ x`, and transcendence is its negation.

```
/-- Algebraicity over `ℤ` is the integer polynomial relation (Mathlib's definition, unfolded). -/
theorem isAlgebraic_int_iff (x : ℝ) :
    IsAlgebraic ℤ x ↔ ∃ p : ℤ[X], p ≠ 0 ∧ aeval x p = 0 := Iff.rfl
```

Kernel report: `isAlgebraic_int_iff`: propext, Classical.choice, Quot.sound

Lindemann's theorem that π is transcendental is not available as a Lean proof in the library used here. It enters as the explicit hypothesis `LindemannPi : Transcendental ℤ π` Premise; the class-level consequences below are proved from it.

Theorem 18.1 Proved from premise**.** *Assume π is transcendental. Then for every $p \in \mathbb{Z}[X] \setminus \{0\}$, $p(\pi) \neq 0$. Equivalently, $\{p \in \mathbb{Z}[X] \setminus \{0\} : p(\pi) = 0\} = \varnothing$.*

```
variable (LindemannPi : Transcendental ℤ π)
include LindemannPi

/-- No nonzero integer polynomial vanishes at `π`. -/
theorem pi_poly_relation_null : ∀ p : ℤ[X], p ≠ 0 → aeval π p ≠ 0 :=
  fun p hp h => LindemannPi ⟨p, hp, h⟩

/-- The polynomial-relation solution set for `π` is empty. -/
theorem pi_poly_relation_set_empty : {p : ℤ[X] | p ≠ 0 ∧ aeval π p = 0} = ∅ :=
  Set.eq_empty_of_forall_notMem fun p h => pi_poly_relation_null LindemannPi p h.1 h.2
```

Kernel report: `pi_poly_relation_null`: propext, Classical.choice, Quot.sound; `pi_poly_relation_set_empty`: propext, Classical.choice, Quot.sound

R4 checks the same contract for e, with the transcendence of e over $\mathbb{Q}$ as its hypothesis Reused:

```
theorem e_algebraic_class_closure_contract (h : Transcendental ℚ (Real.exp 1)) :
    ∀ p : Polynomial ℚ, p ≠ 0 → Polynomial.aeval (Real.exp 1) p ≠ 0 := ...
```

Kernel report: `e_algebraic_class_closure_contract`: propext, Classical.choice, Quot.sound

This enlarges the closed class: "no nontrivial linear integer relation closes on π" becomes "no nontrivial finite polynomial integer relation closes on π." The enlargement is itself a theorem:

Proposition 18.2 (The linear class embeds in the polynomial class) Proved**.** *The map $(a, b) \mapsto aX - b$ is injective from $\mathbb{Z}^2$ to $\mathbb{Z}[X]$, sends pairs with $a \neq 0$ to nonzero polynomials, and sends solutions of $ax - b = 0$ to roots: $(aX - b)(x) = ax - b$. Hence, for every real x, if x is transcendental then $ax - b \neq 0$ for all integers $a \neq 0$, b.*

```
/-- The linear relation `a X - b`. -/
noncomputable def linPoly (a b : ℤ) : ℤ[X] := C a * X - C b

theorem linPoly_coeff_one (a b : ℤ) : (linPoly a b).coeff 1 = a := by
  rw [linPoly, coeff_sub, coeff_C_mul_X, coeff_C]
  simp

theorem linPoly_coeff_zero (a b : ℤ) : (linPoly a b).coeff 0 = -b := by
  rw [linPoly, coeff_sub, coeff_C_mul_X, coeff_C]
  simp

theorem linPoly_ne_zero {a b : ℤ} (ha : a ≠ 0) : linPoly a b ≠ 0 := fun h =>
  ha (by rw [← linPoly_coeff_one a b, h, coeff_zero])

theorem aeval_linPoly (x : ℝ) (a b : ℤ) : aeval x (linPoly a b) = (a : ℝ) * x - b := by
  simp [linPoly]

/-- The linear relation class embeds in the polynomial relation class. -/
theorem linPoly_injective : Function.Injective (fun ab : ℤ × ℤ => linPoly ab.1 ab.2) := by
  rintro ⟨a, b⟩ ⟨a', b'⟩ h
  have h1 := congrArg (fun p : ℤ[X] => p.coeff 1) h
  have h0 := congrArg (fun p : ℤ[X] => p.coeff 0) h
  simp only [linPoly_coeff_one, linPoly_coeff_zero, neg_inj] at h1 h0
  rw [h1, h0]

theorem linear_class_into_poly_class {x : ℝ} {a b : ℤ} (h : a ≠ 0 ∧ (a : ℝ) * x - b = 0) :
    linPoly a b ≠ 0 ∧ aeval x (linPoly a b) = 0 :=
  ⟨linPoly_ne_zero h.1, (aeval_linPoly x a b).trans h.2⟩

/-- Closing the polynomial class closes the linear class. -/
theorem int_relation_null_of_transcendental {x : ℝ} (hx : Transcendental ℤ x) :
    ∀ a b : ℤ, a ≠ 0 → (a : ℝ) * x - b ≠ 0 :=
  fun a b ha h => hx ⟨linPoly a b, linear_class_into_poly_class ⟨ha, h⟩⟩
```

Kernel report: `linPoly_coeff_one`: propext, Classical.choice, Quot.sound; `linPoly_coeff_zero`: propext, Classical.choice, Quot.sound; `linPoly_ne_zero`: propext, Classical.choice, Quot.sound; `aeval_linPoly`: propext, Classical.choice, Quot.sound; `linPoly_injective`: propext, Classical.choice, Quot.sound; `linear_class_into_poly_class`: propext, Classical.choice, Quot.sound; `int_relation_null_of_transcendental`: propext, Classical.choice, Quot.sound

So Theorem 17.2 also follows from Lindemann's theorem. In this paper it is obtained from Mathlib's proof of irrationality, independently of the Lindemann premise.

Again, the proof does not reside in inspecting the decimal expansion of π, nor in testing polynomial after polynomial. The theorem closes the defined relation class. Once established, the result may be reused as a premise. By Theorem 1, such reuse is deductively conservative relative to assumptions that already prove it.

19. Gödel as the complementary form

Lindemann supplies a class-level closure: $\forall p \neq 0,\ p(\pi) \neq 0$. Gödel–Rosser supplies, under its stated hypotheses on a formal theory T, a different kind of class-level theorem: there exists a sentence G_T such that $T \nvdash G_T$ and $T \nvdash \neg G_T$ Premise. In the Lean statement it is the hypothesis `rosser : Indep T G`.

Corollary 19.1 Proved from premise**.** *Let $\vdash$ be sound and complete for a semantics obeying the Tarski clause, and let $T \nvdash G$ and $T \nvdash \neg G$. Then G holds in some model of T and fails in another; $T + G$ and $T + \neg G$ are consistent; and $T + G \vdash G$, $T + \neg G \vdash \neg G$.*

```
/-- §19: a Gödel–Rosser sentence for `T`, supplied as an explicit hypothesis, splits the models
of `T` and is decided in either extension. -/
theorem rosser_instance (hsound : Sound Sat) (hcomp : Complete Sat) (hneg : NegClause Sat)
    {T : Set Sentence} {G : Sentence} (rosser : Indep T G) :
    ((∃ M, ModelsAll Sat M T ∧ Sat M G) ∧ (∃ N, ModelsAll Sat N T ∧ Sat N (neg G))) ∧
      (Consistent (T ∪ {G}) ∧ Prov (T ∪ {G}) G) ∧
      (Consistent (T ∪ {neg G}) ∧ Prov (T ∪ {neg G}) (neg G)) :=
  ⟨model_split hcomp hneg rosser, indep_decided_in_extensions hsound hcomp hneg rosser⟩
```

Kernel report: `rosser_instance`: propext, Classical.choice, Quot.sound

The corpus contains a checked route to such a sentence. The diagonal bridge R6 [18] constructs a sentence from explicit coding receipts, namely quotation, substitution and a formula expressing non-provability, and proves that it has the diagonal meaning (`constructed_sentence_has_diagonal_meaning`) Reused. In its own words it is "a checked bridge schema, not an arithmetization of a named theory".

Corollary 19.2 Proved from reused; premise**.** *Let $\vdash$ be sound for a semantics obeying the Tarski clause, let M be a model of T, and let G satisfy $M \models G$ if and only if $T \nvdash G$. Then:*

(a) $T \nvdash G$ and $T \nvdash \neg G$;

(b) the same holds for the sentence constructed from any R6 coding interface whose provability is derivability from T and whose truth is satisfaction in M;

(c) the rule "if $T \nvdash \psi$ then $M \not\models \psi$" fails.

Proof. (a) If $T \vdash G$, soundness gives $M \models G$, and the diagonal meaning then gives $T \nvdash G$, a contradiction. So $T \nvdash G$, and $M \models G$ by the diagonal meaning. If $T \vdash \neg G$, soundness and the Tarski clause give $M \not\models G$, a contradiction. These are R6's `semantic_diagonal_is_unprovable` and `semantic_diagonal_negation_is_unprovable`. (b) R6's construction supplies the diagonal meaning. (c) G is not derivable and is true in M (R6 `unprovability_does_not_imply_falsity`). □

```
/-- Corollary 19.2(a). If, in a model `M` of `T`, a sentence `G` says of itself that `T` does not
derive it, then soundness and the negation clause make `G` independent of `T`
(R6 `semantic_diagonal_is_unprovable`, `semantic_diagonal_negation_is_unprovable`). -/
```

```
theorem diagonal_indep (hsound : Sound Sat) (hneg : NegClause Sat) {T : Set Sentence}
    {M : Model} (hM : ModelsAll Sat M T) {G : Sentence} (diag : Sat M G ↔ ¬ Prov T G) :
    Indep T G :=
  ⟨DiagonalSeamBridge.semantic_diagonal_is_unprovable (Prov T) (Sat M) G diag
      fun h => hsound T G h M hM,
   DiagonalSeamBridge.semantic_diagonal_negation_is_unprovable (Prov T) (Sat M) neg G diag
      (fun h => hsound T G h M hM) (hneg M G) fun h => hsound T (neg G) h M hM⟩

/-- Corollary 19.2(b). The same from the coding receipts of the R6 interface: for a
`DiagonalInterpretation` whose provability is derivability from `T` and whose truth is
satisfaction in a model of `T`, the constructed sentence is independent of `T`
(R6 `constructed_sentence_has_diagonal_meaning`). -/
theorem constructed_sentence_indep (hsound : Sound Sat) (hneg : NegClause Sat)
    {T : Set Sentence} {M : Model} (hM : ModelsAll Sat M T) {Code Unary : Type*}
    (frame : DiagonalSeamBridge.DiagonalInterpretation Sentence Code Unary)
    (hholds : frame.holds = Sat M) (hprov : frame.provable = Prov T) :
    Indep T (DiagonalSeamBridge.constructedGodelSentence frame) := by
  have d := DiagonalSeamBridge.constructed_sentence_has_diagonal_meaning frame
  rw [hholds, hprov] at d
  exact diagonal_indep hsound hneg hM d

/-- Corollary 19.2(c). In such a model, not being derivable from `T` does not imply being false
(R6 `unprovability_does_not_imply_falsity`). -/
theorem underivable_not_false (hsound : Sound Sat) {T : Set Sentence} {M : Model}
    (hM : ModelsAll Sat M T) {G : Sentence} (diag : Sat M G ↔ ¬ Prov T G) :
    ¬ ∀ s, ¬ Prov T s → ¬ Sat M s :=
  DiagonalSeamBridge.unprovability_does_not_imply_falsity (Prov T) (Sat M) G diag
    fun h => hsound T G h M hM
```

Kernel report: `diagonal_indep`: none; `constructed_sentence_indep`: propext; `underivable_not_false`: none

Corollary 19.3 Proved from reused; premise **.** *Under the hypotheses of Corollary 19.2, with completeness, the conclusions of Corollary 19.1 hold for G.*

```
/-- Corollary 19.3. The conclusions of Corollary 19.1 hold for such a sentence, with the
Gödel–Rosser hypothesis replaced by the diagonal certificate. -/
theorem diagonal_instance (hsound : Sound Sat) (hcomp : Complete Sat) (hneg : NegClause Sat)
    {T : Set Sentence} {M : Model} (hM : ModelsAll Sat M T) {G : Sentence}
    (diag : Sat M G ↔ ¬ Prov T G) :
    ((∃ M, ModelsAll Sat M T ∧ Sat M G) ∧ (∃ N, ModelsAll Sat N T ∧ Sat N (neg G))) ∧
      (Consistent (T ∪ {G}) ∧ Prov (T ∪ {G}) G) ∧
      (Consistent (T ∪ {neg G}) ∧ Prov (T ∪ {neg G}) (neg G)) :=
  rosser_instance hsound hcomp hneg (diagonal_indep hsound hneg hM diag)
```

Kernel report: `diagonal_instance`: propext, Classical.choice, Quot.sound

Corollary 19.3 replaces the Gödel–Rosser hypothesis of Corollary 19.1 by soundness for one model of T and the diagonal meaning of G in it: the semantic form of Gödel's first theorem. The Gödel–Rosser form needs only consistency and remains a premise. Whether a given effective theory admits such a coding interface is the arithmetization step, which neither this paper nor R6 proves.

The theorem does not say that G_T is "unknowable forever." It establishes a precise limitation of derivability from T. A suitable extension may decide the sentence. What survives recursively is not a magical object called uncertainty. What survives is the requirement that every promoted conclusion be warranted relative to explicitly stated assumptions, semantics, and inferential rules.

20. Application to the Rule 30 center problem

Definition 20.1 (The Rule 30 center sequence). Start from one black cell at position 0: $c(0, x) = 1$ if $x = 0$ and 0 otherwise. Update by

$$c(t+1, x) = c(t, x-1) \text{ XOR } \big(c(t, x) \text{ OR } c(t, x+1)\big).$$

The center sequence is $Q(t) = c(t, 0)$.

This is the object of the existing Rule 30 work, and it is imported rather than redefined. The Lean module `Rule30Bridge` (Rule 30 Bridge R1, 23 October 2026; Claude, with Lu Semita, building on the R2 packet) and its dependencies `Rule30Application` and `SeamClosure` (the R2 packet, developed with ChatGPT) are taken from the R8 packet *Closure at the Adjudication Seam* (24 October 2026) in the corpus repository; Appendix C gives their provenance, hashes and build. Their definitions of the object, and the existing tail predicates for eventual periodicity, are:

```
/-- One black cell at position 0; the new value is left xor F(center, right). -/
def cellF (F : Bool → Bool → Bool) : Nat → Int → Bool
  | 0, x => decide (x = 0)
  | t + 1, x => Bool.xor (cellF F t (x - 1)) (F (cellF F t x) (cellF F t (x + 1)))

def or30 : Bool → Bool → Bool := fun m r => m || r

def cell30 : Nat → Int → Bool := cellF or30

/-- The center column. -/
def center30 (t : Nat) : Bool := cell30 t 0
```

```
def EventuallyPeriodic (c : Nat → Bool) : Prop :=
  ∃ N p, 0 < p ∧ ∀ t, N ≤ t → c (t + p) = c t
```

Definition 20.2 (Eventual-period certificates). For a Boolean sequence Q, let

$$E_Q = \{(P, T) : P > 0 \ \wedge \ \forall t \geq T, \ Q(t+P) = Q(t)\}.$$

Then the alternatives become exact set-theoretic statements:

A (eventual periodicity) $E_Q \neq \varnothing$; in the Lean source, A is stated as nonemptiness of E_Q, which is equivalent to $E_Q \neq \varnothing$ by classical logic;

B (non-eventual-periodicity) $E_Q = \varnothing$.

```
/-- `E Q = {(P, T) | P > 0 ∧ ∀ t ≥ T, Q (t + P) = Q t}`. -/
def E (Q : ℕ → Bool) : Set (ℕ × ℕ) := {PT | 0 < PT.1 ∧ ∀ t, PT.2 ≤ t → Q (t + PT.1) = Q t}

/-- A: eventual periodicity, `E Q ≠ ∅` in the form `E Q` is nonempty. -/
def A (Q : ℕ → Bool) : Prop := (E Q).Nonempty

/-- B: non-eventual-periodicity, `E Q = ∅`. -/
def B (Q : ℕ → Bool) : Prop := E Q = ∅
```

Proposition 20.3 (Expanded forms) Proved.

A holds iff $\exists P > 0 \ \exists T \ \forall t \geq T, \ Q(t+P) = Q(t)$ *(no axioms), and A holds iff* $E_Q \neq \varnothing$ *(classical).*

B holds iff $\forall P > 0\ \forall T\ \exists t \geq T,\ Q(t+P) \neq Q(t)$. In membership form, "no pair lies in E_Q" holds iff $\forall P > 0\ \forall T\ \neg\forall t \geq T,\ Q(t+P) = Q(t)$; this equivalence uses no axioms.

```
/-- A in the form `E Q ≠ ∅` (classical). -/
theorem A_iff_ne_empty (Q : ℕ → Bool) : A Q ↔ E Q ≠ ∅ := Set.nonempty_iff_ne_empty

/-- Expanded form of A. -/
theorem A_iff (Q : ℕ → Bool) :
    A Q ↔ ∃ P, 0 < P ∧ ∃ T, ∀ t, T ≤ t → Q (t + P) = Q t :=
  ⟨fun ⟨⟨P, T⟩, hP, h⟩ => ⟨P, hP, T, h⟩, fun ⟨P, hP, T, h⟩ => ⟨⟨P, T⟩, hP, h⟩⟩

/-- Expanded form of B, membership version (constructive). -/
theorem B_mem_iff (Q : ℕ → Bool) :
    (∀ PT, PT ∉ E Q) ↔ ∀ P, 0 < P → ∀ T, ¬ ∀ t, T ≤ t → Q (t + P) = Q t :=
  ⟨fun h P hP T hT => h ⟨P, T⟩ ⟨hP, hT⟩, fun h ⟨P, T⟩ ⟨hP, hT⟩ => h P hP T hT⟩

/-- Expanded form of B (classical). -/
theorem B_iff (Q : ℕ → Bool) :
    B Q ↔ ∀ P, 0 < P → ∀ T, ∃ t, T ≤ t ∧ Q (t + P) ≠ Q t := by
  rw [B, Set.eq_empty_iff_forall_notMem, B_mem_iff]
  refine forall_congr' fun P => forall_congr' fun _ => forall_congr' fun T => ?_
  push Not
  rfl

/-- A and B for the Rule 30 center sequence, expanded. -/
theorem A_Q_iff : A Q ↔ ∃ P, 0 < P ∧ ∃ T, ∀ t, T ≤ t → Q (t + P) = Q t := A_iff Q

theorem B_Q_iff : B Q ↔ ∀ P, 0 < P → ∀ T, ∃ t, T ≤ t ∧ Q (t + P) ≠ Q t := B_iff Q
```

Kernel report: `A_iff_ne_empty`: propext, Classical.choice, Quot.sound; `A_iff`: none; `B_mem_iff`: none; `B_iff`: propext, Classical.choice, Quot.sound; `A_Q_iff`: none; `B_Q_iff`: propext, Classical.choice, Quot.sound

Proposition 20.4 (The same classes in the existing tail predicates) Proved**.** *Every Boolean sequence Q: A holds iff `EventuallyPeriodic Q` (no axioms); "no pair lies in E_Q" holds iff `NegativeTranslation Q` (no axioms); B holds iff `NonPeriodic Q` (classical); and B holds iff A fails.*

```
def NonPeriodic (c : Nat → Bool) : Prop :=
  ∀ N p, 0 < p → ∃ t, N ≤ t ∧ c (t + p) ≠ c t

def NegativeTranslation (c : Nat → Bool) : Prop :=
  ∀ N p, 0 < p → ¬¬ (∃ t, N ≤ t ∧ c (t + p) ≠ c t)
```

```
/-- A is the existing predicate `EventuallyPeriodic` (constructive). -/
theorem A_iff_eventuallyPeriodic (Q : ℕ → Bool) : A Q ↔ Rule30Application.EventuallyPeriodic Q :=
  ⟨fun ⟨⟨P, T⟩, hP, h⟩ => ⟨T, P, hP, h⟩, fun ⟨T, P, hP, h⟩ => ⟨⟨P, T⟩, hP, h⟩⟩

/-- The membership form of B is the existing predicate `NegativeTranslation` (constructive). -/
theorem B_mem_iff_negativeTranslation (Q : ℕ → Bool) :
    (∀ PT, PT ∉ E Q) ↔ Rule30Application.NegativeTranslation Q :=
  ⟨fun h N p hp hn => h (p, N) ⟨hp, fun t ht =>
      Decidable.byContradiction fun hne => hn ⟨t, ht, hne⟩⟩,
   fun h ⟨P, T⟩ ⟨hP, hper⟩ => h T P hP fun ⟨t, ht, hne⟩ => hne (hper t ht)⟩

/-- B is the existing predicate `NonPeriodic` (classical). -/
theorem B_iff_nonPeriodic (Q : ℕ → Bool) : B Q ↔ Rule30Application.NonPeriodic Q :=
  (B_iff Q).trans ⟨fun h N p hp => h p hp N, fun h P hP T => h T P hP⟩
```

```
/-- B is the negation of A. -/
theorem B_iff_not_A (Q : ℕ → Bool) : B Q ↔ ¬ A Q := Set.not_nonempty_iff_eq_empty.symm
```

Kernel report: `A_iff_eventuallyPeriodic`: none; `B_mem_iff_negativeTranslation`: none; `B_iff_nonPeriodic`: propext, Classical.choice, Quot.sound; `B_iff_not_A`: propext, Quot.sound

The existing predicates already keep the positive form, the witness-bearing negative form and its double-negated constructive variant apart; Proposition 20.4 identifies each with the corresponding statement about E_Q.

Proposition 20.5 Proved. *A and B exclude each other. Classically, A or B holds.*

```
/-- A and B exclude each other (constructive). -/
theorem not_A_and_B (Q : ℕ → Bool) : ¬ (A Q ∧ B Q) := fun ⟨⟨x, hx⟩, hB⟩ =>
  (show x ∈ (∅ : Set (ℕ × ℕ)) from hB ▸ hx)

/-- A or B (classical): the unresolved status is not a third truth value. -/
theorem A_or_B (Q : ℕ → Bool) : A Q ∨ B Q :=
  (Set.eq_empty_or_nonempty (E Q)).elim Or.inr Or.inl
```

Kernel report: `not_A_and_B`: none; `A_or_B`: propext, Classical.choice, Quot.sound

C — current unresolved status Open. No accepted derivation presently closes either $E_Q \neq \varnothing$ or $E_Q = \varnothing$ for the Rule 30 center sequence; this paper proves neither. C is not a third truth value (Proposition 20.5). It is the status of the adjudication: Open_R for the registry of accepted receipts. No claim of formal independence is made, and none follows from openness (Proposition 6.4).

A proof of A need not enumerate the infinite tail. It may supply a finite structural theorem that handles arbitrary $t \geq T$. A proof of B need not enumerate all candidate periods. It may supply a finite structural theorem that, for arbitrary P, T, produces or guarantees a violating t. Thus A requires class-level positive closure, and B requires class-level negative closure. Finite computation may provide witnesses against particular candidates without thereby closing the quantified class:

Receipt 20.6 Receipt. *For every $P \in \{1,2,3\}$ and $T \in \{0,1,2,3\}$ there is t with $T \leq t \leq 9$ and $Q(t+P) \neq Q(t)$. Hence these twelve candidates are not in E_Q.*

The Boolean check is evaluated by the kernel; its reading as a statement about Q is then proved.

```
/-- A finite witness excludes one candidate. -/
theorem not_mem_E_of_witness {Q : ℕ → Bool} {P T t : ℕ} (ht : T ≤ t)
    (hne : Q (t + P) ≠ Q t) : (P, T) ∉ E Q := fun h => hne (h.2 t ht)

/-- The center sequence `Q(t)` of the stipulated Rule 30 construction. -/
abbrev Q : ℕ → Bool := center30

/-- Finite computational receipt, Boolean form: for every `P ∈ {1, 2, 3}` and `T ∈ {0, 1, 2, 3}`
some `t` with `T ≤ t ≤ 9` has `Q (t + P) ≠ Q t`. -/
def receiptCheck : Bool :=
  (List.range 3).all fun i => (List.range 4).all fun T =>
    (List.range 10).any fun t => decide (T ≤ t) && (Q (t + (i + 1)) != Q t)

/-- The receipt, evaluated by the kernel. -/
theorem receiptCheck_eq_true : receiptCheck = true := by decide
```

```
/-- The receipt read as a statement about `Q`. -/
theorem finite_exclusion_receipt :
    ∀ P T : ℕ, 0 < P → P < 4 → T < 4 → ∃ t, T ≤ t ∧ t < 10 ∧ Q (t + P) ≠ Q t := by
  intro P T hP hP4 hT4
  have h := receiptCheck_eq_true
  simp only [receiptCheck, List.all_eq_true, List.any_eq_true, List.mem_range,
    Bool.and_eq_true, decide_eq_true_eq, bne_iff_ne, ne_eq] at h
  obtain ⟨t, ht10, hTt, hne⟩ := h (P - 1) (by omega) T hT4
  refine ⟨t, hTt, ht10, ?_⟩
  rwa [show P - 1 + 1 = P by omega] at hne

/-- Hence those twelve candidates are not in `E Q`. -/
theorem finite_family_excluded : ∀ P T : ℕ, 0 < P → P < 4 → T < 4 → (P, T) ∉ E Q := by
  intro P T hP hP4 hT4
  obtain ⟨t, ht, _, hne⟩ := finite_exclusion_receipt P T hP hP4 hT4
  exact not_mem_E_of_witness ht hne
```

Kernel report: `not_mem_E_of_witness`: none; `receiptCheck_eq_true`: none; `finite_exclusion_receipt`: propext, Quot.sound; `finite_family_excluded`: propext, Quot.sound

An infinite subfamily of candidates can be closed by theorem rather than by computation. The existing work proves that the center column is never eventually white and never eventually black Reused:

```
/-- The center column is not eventually white: if it were, the right neighbour would only ever switch
on (R(t+1) = R(t) or cell(t,2)), so it would settle too, against Jen's theorem. -/
theorem center30_not_eventually_white : ¬ ∃ M, ∀ t, M ≤ t → center30 t = false := ...

/-- The center column is not eventually black: at a black center cell c(t+1) = not L(t), so the left
neighbour would be eventually white, against Jen's theorem for the columns -1 and 0. -/
theorem center30_not_eventually_black : ¬ ∃ M, ∀ t, M ≤ t → center30 t = true := ...
```

Kernel report: `center30_not_eventually_white`: propext, Quot.sound; `center30_not_eventually_black`: propext, Quot.sound

Theorem 20.7 (All candidates of period 1 are excluded) Proved from reused**.** *For every T, $(1, T) \notin E_Q$.*

Proof. If $(1, T) \in E_Q$, then $Q(t+1) = Q(t)$ for all $t \geq T$, so $Q(t) = Q(T)$ for all $t \geq T$ by induction. If $Q(T) = 0$ the center column is eventually white, and if $Q(T) = 1$ it is eventually black; both are excluded by the reused theorems. □

```
/-- Every candidate of period 1 is excluded: an infinite subfamily of the candidate class, closed
by the reused theorems `center30_not_eventually_white` and `center30_not_eventually_black`. -/
theorem period_one_excluded : ∀ T, (1, T) ∉ E Q := by
  intro T ⟨_, h⟩
  have hconst : ∀ n, Q (T + n) = Q T := by
    intro n
    induction n with
    | zero => rfl
    | succ n ih => exact (h (T + n) (Nat.le_add_right T n)).trans ih
  have hall : ∀ t, T ≤ t → Q t = Q T := fun t ht => by
    have := hconst (t - T)
    rwa [Nat.add_sub_cancel' ht] at this
  cases hQ : Q T with
  | false => exact center30_not_eventually_white ⟨T, fun t ht => (hall t ht).trans hQ⟩
  | true => exact center30_not_eventually_black ⟨T, fun t ht => (hall t ht).trans hQ⟩
```

Kernel report: `period_one_excluded`: propext, Quot.sound

Neither the finite receipt nor the closure of the infinite subfamily $\{(1, T)\}$ closes the class E_Q:

Theorem 20.8 (Finite prefixes do not decide A or B) Proved. *For every Boolean sequence q and every N, there are sequences q_1, q_2 agreeing with q at all times $t < N$, with A for q_1 and B for q_2.*

Proof. Let $q_1(t) = q(t)$ for $t < N$ and $q_1(t) = 0$ otherwise; then $(1, N) \in E_{q_1}$. Let $q_2(t) = q(t)$ for $t < N$ and, for $t \geq N$, $q_2(t) = 1$ exactly when t is a perfect square. Given $P > 0$ and T, put $m = T + N + P$ and $t = m^2 \geq \max(T, N)$. Then $q_2(t) = 1$, while $m^2 < m^2 + P < (m+1)^2$ because $P \leq 2m$, so $q_2(t + P) = 0$. Hence $(P, T) \notin E_{q_2}$. □

Corollary 20.9 Proved. *Some eventually periodic sequence satisfies the statement of Receipt 20.6 (one agreeing with Q at all times below 13). Hence Receipt 20.6 does not entail B.*

```
/-- The indicator of perfect squares. -/
def isSq (t : ℕ) : Bool := Nat.sqrt t * Nat.sqrt t == t

theorem isSq_mul_self (m : ℕ) : isSq (m * m) = true := by
  simp [isSq, Nat.sqrt_eq]

theorem isSq_mul_self_add {m a : ℕ} (h0 : 0 < a) (h : a ≤ m + m) : isSq (m * m + a) = false := by
  simp only [isSq, Nat.sqrt_add_eq m h, beq_eq_false_iff_ne, ne_eq]
  omega

/-- Every finite prefix of every Boolean sequence extends both to a sequence satisfying A and to a
sequence satisfying B. -/
theorem prefix_does_not_decide (Q : ℕ → Bool) (N : ℕ) :
    (∃ Q₁ : ℕ → Bool, (∀ t < N, Q₁ t = Q t) ∧ A Q₁) ∧
      (∃ Q₂ : ℕ → Bool, (∀ t < N, Q₂ t = Q t) ∧ B Q₂) := by
  refine ⟨⟨fun t => if t < N then Q t else false, fun t ht => ite_eq_left ht, (1, N), Nat.one_pos,
    fun t ht => ?_⟩, ⟨fun t => if t < N then Q t else isSq t, fun t ht => ite_eq_left ht, ?_⟩⟩
  · simp only
    rw [ite_eq_right (by omega), ite_eq_right (by omega)]
  · refine Set.eq_empty_of_forall_notMem ?_
    rintro ⟨P, T⟩ ⟨hP, hper⟩
    have hm : T + N + P ≤ (T + N + P) * (T + N + P) := Nat.le_mul_self _
    have h1 := hper ((T + N + P) * (T + N + P)) (by simp only; omega)
    simp only at h1
    rw [ite_eq_right (by omega), ite_eq_right (by omega), isSq_mul_self_add hP (by omega),
      isSq_mul_self] at h1
    exact Bool.false_ne_true h1

/-- The finite receipt does not close B: an eventually periodic sequence satisfies it too. -/
theorem receipt_compatible_with_A : ∃ Q₁ : ℕ → Bool, A Q₁ ∧
    ∀ P T : ℕ, 0 < P → P < 4 → T < 4 → ∃ t, T ≤ t ∧ t < 10 ∧ Q₁ (t + P) ≠ Q₁ t := by
  obtain ⟨⟨Q₁, hpre, hA⟩, _⟩ := prefix_does_not_decide Q 13
  refine ⟨Q₁, hA, fun P T hP hP4 hT4 => ?_⟩
  obtain ⟨t, ht, ht10, hne⟩ := finite_exclusion_receipt P T hP hP4 hT4
  refine ⟨t, ht, ht10, ?_⟩
  rw [hpre _ (by omega), hpre _ (by omega)]
  exact hne
```

Kernel report: `isSq_mul_self`: propext, Classical.choice, Quot.sound; `isSq_mul_self_add`: propext, Classical.choice, Quot.sound; `prefix_does_not_decide`: propext, Classical.choice, Quot.sound; `receipt_compatible_with_A`: propext, Classical.choice, Quot.sound

R4 checks the same structure for its own windows: every finite window has one extension with the all-stages property and one without (`finite_window_has_opposite_extensions`), and no finite prefix certifies permanence (`no_finite_prefix_certificate_of_permanence`) Reused.

Proposition 20.10 (Closing a subfamily is not closing the class) Proved**.** *The sequence* $q(t) = t \bmod 2$ *has no certificate of period* 1*, yet* $(2, 0) \in E_q$.

```
/-- Excluding an infinite subfamily does not close the class: a sequence with no certificate of
period 1 can still be eventually periodic. -/
theorem period_one_closure_not_full :
    ∃ q : ℕ → Bool, (∀ T, (1, T) ∉ E q) ∧ A q := by
  refine (fun t => t % 2 == 1, fun T h => ?_, (2, 0), Nat.two_pos, fun t _ => ?_⟩
  · have h1 := h.2 T (Nat.le_refl T)
    simp only at h1
    rcases Nat.mod_two_eq_zero_or_one T with h0 | h0
    · rw [Nat.add_mod, h0] at h1; simp at h1
    · rw [Nat.add_mod, h0] at h1; simp at h1
  · simp only
    rw [Nat.add_mod_right]
```

Kernel report: `period_one_closure_not_full`: propext, Classical.choice, Quot.sound

In the existing work, the remaining question is reduced to one implication between adjacent columns Reused:

```
/-- Problem 1 for Rule 30 (as ¬ eventually periodic) is equivalent to one implication: if the center
column is eventually periodic, so is the column just right of it. Constructive in both directions. -/
theorem rule30_problem1_iff_marginal_step :
    ¬ EventuallyPeriodic center30 ↔ (EventuallyPeriodic center30 → EventuallyPeriodic right30) := ...
```

Kernel report: `rule30_problem1_iff_marginal_step`: propext, Quot.sound

Proposition 20.11 Proved from reused**.** *B holds for the Rule 30 center sequence if and only if eventual periodicity of the center column implies eventual periodicity of the column immediately to its right.*

```
/-- B for the Rule 30 center sequence is one implication between adjacent columns; this uses the
reused theorem `rule30_problem1_iff_marginal_step`. -/
theorem B_Q_iff_marginal_step :
    B Q ↔ (Rule30Application.EventuallyPeriodic center30 →
      Rule30Application.EventuallyPeriodic right30) :=
  (B_iff_not_A Q).trans ((not_congr (A_iff_eventuallyPeriodic Q)).trans
    rule30_problem1_iff_marginal_step)
```

Kernel report: `B_Q_iff_marginal_step`: propext, Quot.sound

C remains until one of those quantified relations is closed.

21. The general theorem schema

The entire argument can be compressed to the following schema. Let $(\Gamma, \vdash, \mathcal{I})$ specify explicit assumptions Γ, an inferential relation $\vdash$, and a fixed interpretation $\mathcal{I}$ for the terms currently used. Then:

	Statement	Where proved
1	*Derivation.* $\Gamma \vdash \varphi$ licenses φ under that setting.	Definition 1.1
2	*Reuse.* If $\Gamma \vdash \varphi$, adding φ as a premise does not enlarge deductive closure.	Theorem 1(c); Proposition 3.1
3	*Symmetry.* A claim of $\neg\varphi$ made under the same standard requires its own valid derivation.	Lemma 4.3, Theorem 6
4	*Changed premises.* If a negative derivation requires a premise not derivable from Γ, the argument has changed its assumptions.	Theorems 2, $2'$
5	*Changed semantics.* If old terms are reinterpreted, the argument has changed its interpreted structure.	Example 10.2, Corollary 10.3, Proposition 10.4
6	*Preserved semantic extension.* New distinctions with old meanings fixed preserve all old truth values.	Theorem 4
7	*Unresolved status.* If neither side has an accepted closure, neither side may be promoted solely because the other remains unproved.	Propositions 5.4, 6.3, 6.4; Theorem 7.4(c)
8	*Self-application.* An objection presented under this same proof standard is itself a claim and is subject to the same requirements.	Theorem 6(c), Propositions 5.1–5.3, 12.1

There is no additional logical category in which an adjudicator's unsupported rejection becomes equivalent to a proof.

22. Plain-language statement

A proof begins with things explicitly taken as given. It applies relations whose use has been specified. It ends with something that follows from those givens. Once that result has been proved, it can be reused without proving it from scratch each time. That is ordinary deductive compression. But the compression does not erase where the result came from.

If another person wants to negate the result under the same assumptions and rules, their negation has to meet the same standard. They may find a mistake. They may reject an assumption. They may add a new assumption. They may reinterpret a term. They may introduce a stronger language. They may prove that two representations are or are not structurally equivalent. All of those are legitimate mathematical moves. What they cannot do under one equal proof standard is convert "I have not accepted your proof" into "I have proved its negation."

Likewise, a theorem established under fixed meanings does not become false merely because later language introduces a finer distinction. If the old meanings are retained, the old relation remains. The finer language may then support new propositions that could not previously be expressed. This is extension, not retroactive contradiction.

23. Interpretive note

The formal results above use no vocabulary beyond ordinary mathematics. The title's word *pharmakon* names, after the fact, the two-sided character of Theorem 1(c).

- *Remedy:* proved results may be compressed and reused.
- *Risk:* the conditions under which they entered deductive closure may become tacit and later be mistaken for unconditional authority.

Proposition 3.1 states the risk as a theorem: the bare sentence does not carry the scope under

which it was established, and carrying the scope recovers it. The word was used earlier in the corpus for the same two-sidedness, in *Deflation of Pharmakon-Based Delusion* [13], where one public label carries two incompatible predicates (Proposition 10.4).

Conclusion

The fundamental formal relation is: explicit givens $\longrightarrow$ deductive closure, with $\mathrm{Cn}(\mathrm{Cn}(\Gamma)) = \mathrm{Cn}(\Gamma)$ and therefore

$$\Gamma \vdash \varphi \implies \mathrm{Cn}(\Gamma \cup \{\varphi\}) = \mathrm{Cn}(\Gamma) \qquad \text{(Theorem 1)}.$$

This explains why established knowledge can subsequently function as a given without being reproved. Consistency then supplies the reciprocal constraint: for a consistent Γ, $\Gamma \vdash \varphi \Rightarrow \Gamma \nvdash \neg\varphi$ (Lemma 4.3). And cut gives the stronger adversarial result: $\Gamma \vdash \varphi$ and $\Gamma \cup \Delta \vdash \neg\varphi$ imply that some premise in Δ is genuinely new, unless the original setting is inconsistent or a derivation is invalid (Theorems 2 and 6); moreover $\Gamma \cup \Delta$ is then itself inconsistent (Theorem 2′). Finally, semantic expansion gives: new distinctions with preserved old interpretations preserve old truths (Theorem 4).

These statements jointly establish the desired class-level distinction: what has been proved under fixed relations may be reused as established; what has not been closed remains unresolved; a change of premises, semantics, or proof rules must be made explicit; and an objection is subject to the same proof standard as the claim it attacks. No privileged adjudicating viewpoint is required.

A. Paper statement to Lean declaration

All declarations are in the supplement file `AxiomAsPharmakon.lean`, except those in the namespaces `Rule30Bridge`, `SemanticDeflation`, `LambdaSpatiotemporal`, `UniversalRecoveryClosure`, `ClosureConsequences`, `DiagonalSeamBridge`, `LambdaIII`, `LambdaConstitutive`, `LambdaInstrument`, `LambdaCarriedState`, `EnumerativeEnergy`, `DimensionalLadder`, `UnwitnessableVerdict` and `BeforeTheProof`, which are in the reused files under `reused/`. The rows labelled with earlier works list the imported declarations named in this paper, with the axioms Lean reports for them in this build.

In the Axioms column, ∅ means that Lean reports no axioms, P means `propext`, E means `propext, Quot.sound`, and C means `propext, Classical.choice, Quot.sound`; the exact text is in Appendix B.

Paper	Statement	Lean declaration	Ax.	Status
Lemma 1.2	Monotonicity (derived)	`Consequence.monotonicity`	∅	Proved
Theorem 1(a)	$\Gamma \subseteq \mathrm{Cn}(\Gamma)$	`Consequence.subset_Cn`	∅	Proved
Theorem 1(b)	Idempotence, membership form	`Consequence.mem_Cn_Cn_iff`	∅	Proved
Theorem 1(b)	Idempotence, equality form	`Consequence.closure_idempotent`	E	Proved
Theorem 1(c)	Conservative reuse, membership form	`Consequence.mem_Cn_union_singleton_iff`	∅	Proved
Theorem 1(c)	Conservative reuse, equality form	`Consequence.proved_as_premise_is_conservative`	E	Proved
Theorem 1(c)	Conservative reuse, `insert` form	`Consequence.proved_as_premise_is_conservative_insert`	E	Proved

Paper	Statement	Lean declaration	Ax.	Status
Prop. 3.1(a)	Erasing the scope blocks recovery	`Antecedents.scope_erasure_blocks_recovery`	∅	Proved from reused
Prop. 3.1(b)	Retaining the scope recovers	`Antecedents.scope_retention_recovers`	∅	Proved from reused
Prop. 3.1(c)	The hypotheses of (a) are met	`Antecedents.scope_erasure_instance`	∅	Proved from reused
Prop. 3.1(d)	The sentence does not recover its premise set	`Antecedents.scope_not_recovered_from_sentence`	∅	Proved from reused
Prop. 3.1(e)	Carrying the premise set recovers it	`Antecedents.scope_recovered_when_carried`	∅	Proved from reused
Prop. 3.2(a)	Enlarging the premise set: monotonicity is the bridge	`Antecedents.enlarging_scope_translates`	∅	Proved from reused
Prop. 3.2(b)	Discharging an established premise: Theorem 1(c) is the bridge	`Antecedents.discharging_established_premise_translates`	∅	Proved from reused
Lemma 4.3	No contradictory pair in a consistent set	`Consequence.Consistent.not_both`	∅	Proved
		`Consequence.Consistent.not_prov_neg`	∅	
Theorem 2	General constructive core	`Consequence.not_all_premises_derivable`	∅	Proved
Theorem 2(a)	Constructive form	`Consequence.refutation_requires_new_premise`	∅	Proved
Theorem 2(b)	Existential form	`Consequence.negative_derivation_requires_new_premise`	C	Proved
Theorem 2′	Refuting premise set inconsistent	`Consequence.refuting_extension_inconsistent`	∅	Proved
Prop. 4.4(i)	Consistency needed	`Countermodels.refutation_needs_consistency`	∅	Proved
Prop. 4.4(ii)	Derivation needed	`Countermodels.refutation_needs_derivation`	∅	Proved
Prop. 5.1	Non-acceptance is not refutation	`Consequence.nonacceptance_not_refutation`	∅	Proved
Prop. 5.2	A hedge is not a derivation	`Consequence.hedge_not_refutation`	∅	Proved
Prop. 5.3	Bridge rule forces acceptance	`Consequence.bridge_rule_forces_acceptance`	∅	Proved
Prop. 5.4	Underivable is not refuted	`Countermodels.underivable_not_refuted`	∅	Proved
Prop. 6.3	Independence implies openness	`Consequence.Registry.open_of_indep`	∅	Proved
Prop. 6.4	Openness does not imply independence	`Countermodels.open_not_indep`	∅	Proved
Theorem 3(a)	Model split, constructive	`Consequence.model_split_core_pos`	∅	Proved; premise: completeness
		`Consequence.model_split_core_neg`	∅	
Theorem 3(b)	Model split, existential	`Consequence.model_split`	C	Proved; premise: completeness
Lemma 7.2	Explosion: independence gives consistency	`Consequence.consistent_of_indep`	∅	Proved
Lemma 7.3	Countermodel gives underivability	`Consequence.not_prov_of_countermodel`	∅	Proved; premise: soundness

Paper	Statement	Lean declaration	Ax.	Status
Theorem 7.4(a)	An independent sentence is a seam	`Antecedents.indep_is_seam`	C	Proved; premise: completeness
Theorem 7.4(b)	Truth not recovered from what Γ settles	`Antecedents.indep_truth_not_recovered`	C	Proved from reused; premise: completeness
Theorem 7.4(c)	Sound adjudicators abstain	`Antecedents.indep_forces_abstention`	C	Proved from reused; premise: completeness
Cor. 3.1(a)	$\Gamma + \varphi$ consistent	`Consequence.indep_union_pos_consistent`	∅	Proved; premises: soundness, completeness
Cor. 3.1(b)	$\Gamma + \neg\varphi$ consistent	`Consequence.indep_union_neg_consistent`	C	Proved; premises: soundness, completeness
Cor. 3.1	Decided in either extension	`Consequence.indep_decided_in_extensions`	C	Proved; premises: soundness, completeness
Prop. 8.1	Independence is relative	`Consequence.not_indep_union_self`	∅	Proved
Theorem 4	Terms keep their values	`FO.eval_onTerm`	∅	Proved
		`FO.eval_onTerms`	∅	
Theorem 4	Semantic stability	`FO.realize_onFormula`	∅	Proved
		`FO.realize_onFormula_of_realize`	∅	
Example 10.2	Reinterpretation changes truth	`FO.reinterpretation_changes_truth`	∅	Proved
Cor. 10.3	Preservation hypothesis needed	`FO.reinterpretation_not_expansion`	∅	Proved
Prop. 10.4(a)	Disagreeing readings: no shared definition	`Antecedents.readings_have_no_shared_definition`	∅	Proved from reused
Prop. 10.4(b)	Hence not an expansion	`Antecedents.not_expansion_of_readings`	∅	Proved from reused
Prop. 10.4(c)	Example 10.2 is such a case	`Antecedents.shared_symbol_does_not_preserve_meaning`	∅	Proved from reused
Theorem 5	Terms transported	`FO.Iso.eval_term`	∅	Proved
		`FO.Iso.eval_terms`	∅	
Theorem 5	Isomorphism transport	`FO.Iso.realize`	∅	Proved
		`FO.Iso.realize_comp`	∅	
Prop. 11.2	Injectivity, surjectivity needed	`FO.injectivity_needed`	∅	Proved
		`FO.surjectivity_needed`	∅	
Prop. 11.3	Isomorphisms preserve predicates; transport composes	`Antecedents.iso_preserves_predicate`	∅	Proved from reused
		`Antecedents.iso_transport_composes`	∅	
Theorem 6(a)	Symmetric adjudication, core	`Consequence.symmetric_adjudication_core`	∅	Proved
Theorem 6(b)	Symmetric adjudication, disjunctive	`Consequence.symmetric_adjudication`	C	Proved
Theorem 6(c)	Roles exchanged	`Consequence.symmetric_adjudication_core_swap`	∅	Proved
Prop. 12.1	No refutation free of new premises	`Antecedents.no_complement_free_refutation`	C	Proved from reused

Paper	Statement	Lean declaration	Ax.	Status
Section 13	Not self-sealing: the theorem governs its own challenge	`Consequence.symmetric_adjudication_core_swap`	∅	Proved
Prop. 14.1	Truth under one interpretation is a local warrant	`Antecedents.frame_truth_iff_sample_warrant`	∅	Proved from reused
		`Antecedents.truth_is_local_warrant`	∅	
Prop. 15.1	Null closure, membership form	`ClassClosure.null_closure_mem`	∅	Proved
Prop. 15.1	Null closure, equality form	`ClassClosure.null_closure`	E	Proved
Theorem 16.1	$P_n < 2\pi$; empty equality set	`ClassClosure.P_lt_two_pi`	C	Mathlib
		`ClassClosure.polygon_perimeter_lt`	C	
		`ClassClosure.polygon_equality_set_empty`	C	
Theorem 16.2	Two-sided bound; convergence	`ClassClosure.two_pi_sub_lt_P`	C	Mathlib
		`ClassClosure.abs_two_pi_sub_P_lt`	C	
		`ClassClosure.P_tendsto`	C	
		`ClassClosure.P_approaches_not_attained`	C	
Prop. 17.1	Rational iff integer relation	`ClassClosure.rational_iff_int_relation`	C	Proved
Theorem 17.2	No integer relation for π	`ClassClosure.pi_int_relation_null`	C	Mathlib
		`ClassClosure.pi_int_relation_set_empty`	C	
Theorem 18.1	No polynomial relation for π	`ClassClosure.pi_poly_relation_null`	C	Proved; premise: Lindemann
		`ClassClosure.pi_poly_relation_set_empty`	C	
Prop. 18.2	Linear class embeds in polynomial class	`ClassClosure.linPoly_injective`	C	Proved
		`ClassClosure.linear_class_into_poly_class`	C	
		`ClassClosure.int_relation_null_of_transcendental`	C	
Cor. 19.1	Gödel–Rosser instance	`Consequence.rosser_instance`	C	Proved; premises: Gödel–Rosser, soundness, completeness
Cor. 19.2(a)	A diagonal sentence is independent	`Antecedents.diagonal_indep`	∅	Proved from reused; premise: soundness
Cor. 19.2(b)	From the R6 coding receipts	`Antecedents.constructed_sentence_indep`	P	Proved from reused; premise: soundness
Cor. 19.2(c)	Underivable is not false	`Antecedents.underivable_not_false`	∅	Proved from reused; premise: soundness

Paper	Statement	Lean declaration	Ax.	Status
Cor. 19.3	Cor. 19.1 from the diagonal certificate	`Antecedents.diagonal_instance`	C	Proved from reused; premises: soundness, completeness
Prop. 20.3	Expanded forms of A and B	`EventualPeriod.A_iff_ne_empty`	C	Proved
		`EventualPeriod.A_iff`	∅	
		`EventualPeriod.B_mem_iff`	∅	
		`EventualPeriod.B_iff`	C	
Prop. 20.4	Existing tail predicates	`EventualPeriod.A_iff_eventuallyPeriodic`	∅	Proved
		`EventualPeriod.B_mem_iff_negativeTranslation`	∅	
		`EventualPeriod.B_iff_nonPeriodic`	C	
		`EventualPeriod.B_iff_not_A`	E	
Prop. 20.5	A, B exclusive; A or B	`EventualPeriod.not_A_and_B`	∅	Proved
		`EventualPeriod.A_or_B`	C	
Status C	Which of A, B holds for Rule 30	—	—	Open
Receipt 20.6	Kernel-evaluated check	`Rule30Center.receiptCheck_eq_true`	∅	Receipt
Receipt 20.6	Its reading; twelve candidates excluded	`Rule30Center.finite_exclusion_receipt`	E	Proved
		`Rule30Center.finite_family_excluded`	E	
Reused	Never eventually white or black	`Rule30Bridge.center30_not_eventually_white`	E	Reused
		`Rule30Bridge.center30_not_eventually_black`	E	
Theorem 20.7	Period-1 candidates excluded	`Rule30Center.period_one_excluded`	E	Proved from reused
Theorem 20.8	Finite prefixes do not decide	`EventualPeriod.prefix_does_not_decide`	C	Proved
Cor. 20.9	Receipt compatible with A	`Rule30Center.receipt_compatible_with_A`	C	Proved
Prop. 20.10	Subfamily closure is not class closure	`EventualPeriod.period_one_closure_not_full`	C	Proved
Reused	Problem 1 as one implication	`Rule30Bridge.rule30_problem1_iff_marginal_step`	E	Reused
Prop. 20.11	B as one implication	`Rule30Center.B_Q_iff_marginal_step`	E	Proved from reused
Deflation	Semantic deflation (9 Sept 2026)	`SemanticDeflation.deflation`	∅	Reused
		`SemanticDeflation.constraint_loss`	∅	
		`SemanticDeflation.no_shared_definition`	∅	
		`SemanticDeflation.same_meaning_conflict`	∅	
		`SemanticDeflation.shared_label_does_not_preserve_meaning`	∅	
Part I	Lambda, Reference (10 Sept 2026)	`LambdaSpatiotemporal.postprocessing_cannot_split_fibre`	∅	Reused
		`LambdaSpatiotemporal.target_difference_blocks_factorization`	∅	
		`LambdaSpatiotemporal.complement_requirement_blocks_absolute_priority`	∅	

Paper	Statement	Lean declaration	Ax.	Status
		`LambdaSpatiotemporal.no_first_position`	∅	
		`LambdaSpatiotemporal.local_warrant_not_universal_in_general`	∅	
		`LambdaSpatiotemporal.absolute_label_does_not_self_license`	∅	
		`LambdaSpatiotemporal.same_label_can_hide_different_meanings`	∅	
		`LambdaSpatiotemporal.predicate_preservation_composes`	∅	
		`LambdaSpatiotemporal.translations_need_not_commute`	∅	
		`LambdaSpatiotemporal.same_input_can_receive_different_interpretations`	∅	
R8.1 core	Adjudication seam (24 Oct 2026)	`UniversalRecoveryClosure.no_processing_recovers_erased_distinction`	∅	Reused
		`UniversalRecoveryClosure.no_decoder_recovers_erased_distinction`	∅	
		`ClosureConsequences.universal_class_is_closed`	∅	
		`ClosureConsequences.forced_abstention_at_seam`	∅	
		`ClosureConsequences.identity_readout_recovers`	∅	
		`ClosureConsequences.target_retention_recovers`	∅	
R6	Diagonal / recovery bridge (23 Oct 2026)	`DiagonalSeamBridge.constructed_sentence_has_diagonal_meaning`	P	Reused
		`DiagonalSeamBridge.semantic_diagonal_is_unprovable`	∅	
		`DiagonalSeamBridge.semantic_diagonal_negation_is_unprovable`	∅	
		`DiagonalSeamBridge.unprovability_does_not_imply_falsity`	∅	
		`DiagonalSeamBridge.godel_truth_cannot_be_recovered_from_provability`	P	
R4, Part III	Lambda Part III, in the R4 packet	`LambdaIII.scope_retained_is_recoverable`	∅	Reused
		`LambdaIII.scope_forgotten_is_not_recoverable`	∅	
		`LambdaIII.only_error_retracts`	P	
		`LambdaIII.warrant_lapse_does_not_retract`	P	
		`LambdaIII.complement_requirement_blocks_absolute_priority`	∅	
		`LambdaIII.no_first_position_cyclic`	P	
		`LambdaIII.equal_criteria_forces_equal_verdict`	∅	
		`LambdaIII.equal_standard_trichotomy`	∅	
R4, constitutive	Scope, provenance, warrants	`LambdaConstitutive.conveyance_collision_blocks_target`	∅	Reused
		`LambdaConstitutive.metadescription_cannot_split_equal_messages`	∅	
		`LambdaConstitutive.result_label_cannot_recover_origin`	∅	

Paper	Statement	Lean declaration	Ax.	Status
		`LambdaConstitutive.warranted_reuse_needs_scope_preservation`	∅	
		`LambdaConstitutive.warranted_translation_needs_a_bridge`	∅	
		`LambdaConstitutive.derived_tag_alone_does_not_prove_claim`	∅	
		`LambdaConstitutive.different_origin_rankings_need_no_absolute_comparator`	∅	
R4, instrument	Adjudication instrument	`LambdaInstrument.no_refutation_is_not_a_general_proof`	∅	Reused
		`LambdaInstrument.completeness_licenses_reversal`	C	
		`LambdaInstrument.challenge_is_an_instance`	∅	
		`LambdaInstrument.consideration_has_an_applied_rule`	∅	
		`LambdaInstrument.receipt_does_not_certify_rule_soundness`	∅	
		`LambdaInstrument.finite_window_has_opposite_extensions`	∅	
		`LambdaInstrument.bounded_tests_do_not_prove_candidate`	∅	
		`LambdaInstrument.pi_no_finite_base_fraction`	C	
		`LambdaInstrument.e_algebraic_class_closure_contract`	C	
R4, carried state	Finite prefixes and permanence	`LambdaCarriedState.no_finite_prefix_certificate_of_permanence`	C	Reused
Part II	Lambda Part II (11 Sept 2026)	`EnumerativeEnergy.conservative_representation_preserves_theorems`	∅	Reused
		`EnumerativeEnergy.relabeling_preserves_return`	E	
Dim. Ladder	Dimensional Ladder V2 (9 Sept 2026)	`DimensionalLadder.commuting_bridge_transports_identity`	∅	Reused
		`DimensionalLadder.postprocess_cannot_split`	∅	
		`DimensionalLadder.orientation_invisible_to_position`	∅	
UV, BtP	Unwitnessable Verdict; Before the Proof	`UnwitnessableVerdict.vacuous_pass`	∅	Reused
		`BeforeTheProof.bridge_underdetermination`	∅	

B. Kernel report

The output of the `#print axioms` commands at the end of `AxiomAsPharmakon.lean`, verbatim.

```
'Consequence.monotonicity' does not depend on any axioms
'Consequence.subset_Cn' does not depend on any axioms
'Consequence.mem_Cn_Cn_iff' does not depend on any axioms
'Consequence.closure_idempotent' depends on axioms: [propext, Quot.sound]
'Consequence.union_singleton_derivable' does not depend on any axioms
'Consequence.mem_Cn_union_singleton_iff' does not depend on any axioms
'Consequence.proved_as_premise_is_conservative' depends on axioms: [propext, Quot.sound]
```

```
'Consequence.proved_as_premise_is_conservative_insert' depends on axioms: [propext, Quot.sound]
'Consequence.Contradictory.symm' does not depend on any axioms
'Consequence.contradictory_neg' does not depend on any axioms
'Consequence.Consistent.not_both' does not depend on any axioms
'Consequence.Consistent.not_prov_neg' does not depend on any axioms
'Consequence.not_all_premises_derivable' does not depend on any axioms
'Consequence.refutation_requires_new_premise' does not depend on any axioms
'Consequence.negative_derivation_requires_new_premise' depends on axioms: [propext, Classical.choice, Quot.sound]
'Consequence.refuting_extension_inconsistent' does not depend on any axioms
'Consequence.symmetric_adjudication_core' does not depend on any axioms
'Consequence.symmetric_adjudication' depends on axioms: [propext, Classical.choice, Quot.sound]
'Consequence.symmetric_adjudication_core_swap' does not depend on any axioms
'Consequence.nonacceptance_not_refutation' does not depend on any axioms
'Consequence.hedge_not_refutation' does not depend on any axioms
'Consequence.bridge_rule_forces_acceptance' does not depend on any axioms
'Consequence.Registry.open_of_indep' does not depend on any axioms
'Consequence.not_indep_union_self' does not depend on any axioms
'Consequence.consistent_of_indep' does not depend on any axioms
'Consequence.model_split_core_pos' does not depend on any axioms
'Consequence.model_split_core_neg' does not depend on any axioms
'Consequence.model_split' depends on axioms: [propext, Classical.choice, Quot.sound]
'Consequence.not_prov_of_countermodel' does not depend on any axioms
'Consequence.indep_union_pos_consistent' does not depend on any axioms
'Consequence.indep_union_neg_consistent' depends on axioms: [propext, Classical.choice, Quot.sound]
'Consequence.indep_decided_in_extensions' depends on axioms: [propext, Classical.choice, Quot.sound]
'Consequence.rosser_instance' depends on axioms: [propext, Classical.choice, Quot.sound]
'Countermodels.refutation_needs_consistency' does not depend on any axioms
'Countermodels.refutation_needs_derivation' does not depend on any axioms
'Countermodels.underivable_not_refuted' does not depend on any axioms
'Countermodels.open_not_indep' does not depend on any axioms
'FO.eval_onTerm' does not depend on any axioms
'FO.eval_onTerms' does not depend on any axioms
'FO.realize_onFormula' does not depend on any axioms
'FO.realize_onFormula_of_realize' does not depend on any axioms
'FO.reinterpretation_changes_truth' does not depend on any axioms
'FO.reinterpretation_not_expansion' does not depend on any axioms
'FO.Iso.eval_term' does not depend on any axioms
'FO.Iso.eval_terms' does not depend on any axioms
'FO.Iso.cons_pointwise' does not depend on any axioms
'FO.Iso.toFun_inj' does not depend on any axioms
'FO.Iso.realize' does not depend on any axioms
'FO.Iso.realize_comp' does not depend on any axioms
'FO.twoDistinct_bool' does not depend on any axioms
'FO.twoDistinct_unit' does not depend on any axioms
'FO.injectivity_needed' does not depend on any axioms
'FO.surjectivity_needed' does not depend on any axioms
'ClassClosure.null_closure_mem' does not depend on any axioms
'ClassClosure.null_closure' depends on axioms: [propext, Quot.sound]
'ClassClosure.rational_iff_int_relation' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.pi_int_relation_null' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.pi_int_relation_set_empty' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.isAlgebraic_int_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.pi_poly_relation_null' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.pi_poly_relation_set_empty' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.linPoly_coeff_one' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.linPoly_coeff_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.linPoly_ne_zero' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.aeval_linPoly' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.linPoly_injective' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.linear_class_into_poly_class' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.int_relation_null_of_transcendental' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.P_lt_two_pi' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.polygon_perimeter_lt' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'ClassClosure.polygon_equality_set_empty' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.two_pi_sub_lt_P' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.abs_two_pi_sub_P_lt' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.P_tendsto' depends on axioms: [propext, Classical.choice, Quot.sound]
'ClassClosure.P_approaches_not_attained' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.A_iff_ne_empty' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.A_iff' does not depend on any axioms
'EventualPeriod.B_mem_iff' does not depend on any axioms
'EventualPeriod.B_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.not_A_and_B' does not depend on any axioms
'EventualPeriod.A_or_B' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.not_mem_E_of_witness' does not depend on any axioms
'EventualPeriod.A_iff_eventuallyPeriodic' does not depend on any axioms
'EventualPeriod.B_mem_iff_negativeTranslation' does not depend on any axioms
'EventualPeriod.B_iff_nonPeriodic' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.B_iff_not_A' depends on axioms: [propext, Quot.sound]
'EventualPeriod.period_one_closure_not_full' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.isSq_mul_self' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.isSq_mul_self_add' depends on axioms: [propext, Classical.choice, Quot.sound]
'EventualPeriod.prefix_does_not_decide' depends on axioms: [propext, Classical.choice, Quot.sound]
'Rule30Center.period_one_excluded' depends on axioms: [propext, Quot.sound]
'Rule30Center.B_Q_iff_marginal_step' depends on axioms: [propext, Quot.sound]
'Rule30Center.receiptCheck_eq_true' does not depend on any axioms
'Rule30Center.finite_exclusion_receipt' depends on axioms: [propext, Quot.sound]
'Rule30Center.finite_family_excluded' depends on axioms: [propext, Quot.sound]
'Rule30Center.receipt_compatible_with_A' depends on axioms: [propext, Classical.choice, Quot.sound]
'Rule30Center.A_Q_iff' does not depend on any axioms
'Rule30Center.B_Q_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'Antecedents.scope_erasure_blocks_recovery' does not depend on any axioms
'Antecedents.scope_retention_recovers' does not depend on any axioms
'Antecedents.scope_erasure_instance' does not depend on any axioms
'Antecedents.scope_not_recovered_from_sentence' does not depend on any axioms
'Antecedents.scope_recovered_when_carried' does not depend on any axioms
'Antecedents.enlarging_scope_translates' does not depend on any axioms
'Antecedents.discharging_established_premise_translates' does not depend on any axioms
'Antecedents.indep_is_seam' depends on axioms: [propext, Classical.choice, Quot.sound]
'Antecedents.indep_truth_not_recovered' depends on axioms: [propext, Classical.choice, Quot.sound]
'Antecedents.indep_forces_abstention' depends on axioms: [propext, Classical.choice, Quot.sound]
'Antecedents.readings_have_no_shared_definition' does not depend on any axioms
'Antecedents.not_expansion_of_readings' does not depend on any axioms
'Antecedents.shared_symbol_does_not_preserve_meaning' does not depend on any axioms
'Antecedents.iso_preserves_predicate' does not depend on any axioms
'Antecedents.iso_transport_composes' does not depend on any axioms
'Antecedents.no_complement_free_refutation' depends on axioms: [propext, Classical.choice, Quot.sound]
'Antecedents.frame_truth_iff_sample_warrant' does not depend on any axioms
'Antecedents.truth_is_local_warrant' does not depend on any axioms
'Antecedents.diagonal_indep' does not depend on any axioms
'Antecedents.constructed_sentence_indep' depends on axioms: [propext]
'Antecedents.underivable_not_false' does not depend on any axioms
'Antecedents.diagonal_instance' depends on axioms: [propext, Classical.choice, Quot.sound]
'Rule30Bridge.center30_not_eventually_white' depends on axioms: [propext, Quot.sound]
'Rule30Bridge.center30_not_eventually_black' depends on axioms: [propext, Quot.sound]
'Rule30Bridge.rule30_problem1_iff_marginal_step' depends on axioms: [propext, Quot.sound]
'SemanticDeflation.deflation' does not depend on any axioms
'SemanticDeflation.constraint_loss' does not depend on any axioms
'SemanticDeflation.no_shared_definition' does not depend on any axioms
'SemanticDeflation.same_meaning_conflict' does not depend on any axioms
'SemanticDeflation.shared_label_does_not_preserve_meaning' does not depend on any axioms
'LambdaSpatiotemporal.postprocessing_cannot_split_fibre' does not depend on any axioms
'LambdaSpatiotemporal.target_difference_blocks_factorization' does not depend on any axioms
'LambdaSpatiotemporal.complement_requirement_blocks_absolute_priority' does not depend on any axioms
'LambdaSpatiotemporal.no_first_position' does not depend on any axioms
'LambdaSpatiotemporal.local_warrant_not_universal_in_general' does not depend on any axioms
```

```
'LambdaSpatiotemporal.absolute_label_does_not_self_license' does not depend on any axioms
'LambdaSpatiotemporal.same_label_can_hide_different_meanings' does not depend on any axioms
'LambdaSpatiotemporal.predicate_preservation_composes' does not depend on any axioms
'LambdaSpatiotemporal.translations_need_not_commute' does not depend on any axioms
'LambdaSpatiotemporal.same_input_can_receive_different_interpretations' does not depend on any axioms
'UniversalRecoveryClosure.no_processing_recovers_erased_distinction' does not depend on any axioms
'UniversalRecoveryClosure.no_decoder_recovers_erased_distinction' does not depend on any axioms
'ClosureConsequences.universal_class_is_closed' does not depend on any axioms
'ClosureConsequences.forced_abstention_at_seam' does not depend on any axioms
'ClosureConsequences.identity_readout_recovers' does not depend on any axioms
'ClosureConsequences.target_retention_recovers' does not depend on any axioms
'DiagonalSeamBridge.constructed_sentence_has_diagonal_meaning' depends on axioms: [propext]
'DiagonalSeamBridge.semantic_diagonal_is_unprovable' does not depend on any axioms
'DiagonalSeamBridge.semantic_diagonal_negation_is_unprovable' does not depend on any axioms
'DiagonalSeamBridge.unprovability_does_not_imply_falsity' does not depend on any axioms
'DiagonalSeamBridge.godel_truth_cannot_be_recovered_from_provability' depends on axioms: [propext]
'LambdaIII.scope_retained_is_recoverable' does not depend on any axioms
'LambdaIII.scope_forgotten_is_not_recoverable' does not depend on any axioms
'LambdaIII.only_error_retracts' depends on axioms: [propext]
'LambdaIII.warrant_lapse_does_not_retract' depends on axioms: [propext]
'LambdaIII.complement_requirement_blocks_absolute_priority' does not depend on any axioms
'LambdaIII.no_first_position_cyclic' depends on axioms: [propext]
'LambdaIII.equal_criteria_forces_equal_verdict' does not depend on any axioms
'LambdaIII.equal_standard_trichotomy' does not depend on any axioms
'LambdaConstitutive.conveyance_collision_blocks_target' does not depend on any axioms
'LambdaConstitutive.metadescription_cannot_split_equal_messages' does not depend on any axioms
'LambdaConstitutive.result_label_cannot_recover_origin' does not depend on any axioms
'LambdaConstitutive.warranted_reuse_needs_scope_preservation' does not depend on any axioms
'LambdaConstitutive.warranted_translation_needs_a_bridge' does not depend on any axioms
'LambdaConstitutive.derived_tag_alone_does_not_prove_claim' does not depend on any axioms
'LambdaConstitutive.different_origin_rankings_need_no_absolute_comparator' does not depend on any axioms
'LambdaInstrument.no_refutation_is_not_a_general_proof' does not depend on any axioms
'LambdaInstrument.completeness_licenses_reversal' depends on axioms: [propext, Classical.choice, Quot.sound]
'LambdaInstrument.challenge_is_an_instance' does not depend on any axioms
'LambdaInstrument.consideration_has_an_applied_rule' does not depend on any axioms
'LambdaInstrument.receipt_does_not_certify_rule_soundness' does not depend on any axioms
'LambdaInstrument.finite_window_has_opposite_extensions' does not depend on any axioms
'LambdaInstrument.bounded_tests_do_not_prove_candidate' does not depend on any axioms
'LambdaInstrument.pi_no_finite_base_fraction' depends on axioms: [propext, Classical.choice, Quot.sound]
'LambdaInstrument.e_algebraic_class_closure_contract' depends on axioms: [propext, Classical.choice, Quot.sound]
'LambdaCarriedState.no_finite_prefix_certificate_of_permanence' depends on axioms: [propext,
 Classical.choice,
 Quot.sound]
'EnumerativeEnergy.conservative_representation_preserves_theorems' does not depend on any axioms
'EnumerativeEnergy.relabeling_preserves_return' depends on axioms: [propext, Quot.sound]
'DimensionalLadder.commuting_bridge_transports_identity' does not depend on any axioms
'DimensionalLadder.postprocess_cannot_split' does not depend on any axioms
'DimensionalLadder.orientation_invisible_to_position' does not depend on any axioms
'UnwitnessableVerdict.vacuous_pass' does not depend on any axioms
'BeforeTheProof.bridge_underdetermination' does not depend on any axioms
```

C. Source and receipt notes

Receipt.

- Toolchain: Lean 4.35.0 (`x86_64-unknown-linux-gnu`, commit `eb6cafb7290e8bd5f2937522d8d804f58f02bafc`). Library: Mathlib v4.35.0, commit `56a005855ea17e0b0958f73ec801ae140b0a1679`.
- Main file `AxiomAsPharmakon.lean`: 179 declarations, 120 theorems; no `sorry`, `admit`, `native_decide` or `axiom` declaration; compiled with no errors and no warnings, the only

output being the kernel report of Appendix B. SHA-256 `6fd77d9613d6666928a424ad261614fdd4d05f22eb9cbe56b4c39d1e06d66f13`.

- Reused modules, 29 files under `reused/`, compiled again here with Lean 4.35.0 and no errors. From the PDFs of [13] and [14]: `SemanticDeflation` and `LambdaSpatiotemporalFrame`, unchanged (the first matches the source hash recorded in its own receipt; the second matches the byte and line counts of its accepted source). From the R8 packet [18], byte-identical in R8.1: `UniversalRecoveryClosure`, `ClosureConsequences` and the sixteen R6 modules ending in `DiagonalSeamBridge`. From the R4 instrument [12]: `LambdaPartIII`, `LambdaConstitutive`, `LambdaInstrument`, `LambdaCarriedState`, `LambdaInstitutionalSignals`, `LambdaPartII` and `UnwitnessableVerdict_Audit`, whose source hashes match those recorded in the packet; and `DimensionalLadder` [17] and `BeforeTheProof_BU_FINAL` [22]. Two kinds of change: in the seven modules that import Mathlib, `import Mathlib` is replaced by eight named Mathlib modules, since the full library is not built here; in `Rule30Bridge.lean`, five proof steps `simpa [...] using h` become `exact h`, as in the first edition. No declaration is changed. For the 245 theorems of the five R4 modules, `LambdaPartII` and `UnwitnessableVerdict_Audit`, the axioms Lean reports here equal those recorded in the R4 instrument. Hashes, per-module results and the diffs are in `RECEIPT.txt`.

Imported rather than reproved.

- From Mathlib (Lean proofs exist and are used as cited): `irrational_pi` (irrationality of π; Mathlib formalizes Cartwright's proof); `Real.sin_lt` ($\sin x < x$ for $x > 0$); `Real.sin_gt_sub_cube` ($x - x^3/6 < \sin x$ for $x > 0$); `tendsto_const_div_atTop_nhds_zero_nat`; and the library's definitions of sets, real numbers, π, sin, integer polynomials, `IsAlgebraic`, `Transcendental` and `Irrational`.
- Explicit hypotheses (established theorems without a Lean proof in this build): Gödel's completeness theorem (`hcomp : Complete Sat`); the soundness theorem (`hsound : Sound Sat`); the Gödel–Rosser theorem (`rosser : Indep T G`), for Corollary 19.1 only, since Corollaries 19.2–19.3 replace it by the R6 coding receipts and soundness for one model; Lindemann's theorem for π (`LindemannPi : Transcendental ℤ π`). No axiom is declared in the source.
- Reused from earlier corpus work (imported, checked again in this build, not reproved): the module `Rule30Bridge` (Claude, with Lu Semita), with `Rule30Application` and `SeamClosure` (R2 packet, with ChatGPT); used here are the object `Rule30Bridge.center30`, the predicates `EventuallyPeriodic`, `NonPeriodic`, `NegativeTranslation`, and the theorems `center30_not_eventually_white`, `center30_not_eventually_black` and `rule30_problem1_iff_marginal_step`.
- Also reused in this edition (imported, checked again, not reproved), with the declarations listed in the section on prior formal foundations: `SemanticDeflation` (Lu Semita); `LambdaSpatiotemporalFrame` (Lu Semita); `UniversalRecoveryClosure` and `ClosureConsequences` (R8.1 core, with ChatGPT); `DiagonalSeamBridge` and the R6 modules it imports (with ChatGPT; the Rule 30 modules among them are those of [18]); from the R4 packet [12], `LambdaPartIII`, `LambdaConstitutive`, `LambdaInstrument`, `LambdaCarriedState` and `LambdaInstitutionalSignals`, with `LambdaPartII` and `UnwitnessableVerdict_Audit`; `DimensionalLadder` [17] and `BeforeTheProof_BU_FINAL` [22]. In the seven of these that import Mathlib, `import Mathlib` is replaced by eight named Mathlib modules, because the full library is not built here; nothing else in them changes.
- Proved directly: everything else, including the consequence-relation results, the first-order semantics, Theorems 4 and 5, and all countermodels. Lemma 1.2 (monotonicity), assumed in the original formulation, is derived.

References

[1] K. Gödel, Die Vollständigkeit der Axiome des logischen Funktionenkalküls, *Monatshefte für Mathematik und Physik* 37 (1930), 349–360.

[2] K. Gödel, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, *Monatshefte für Mathematik und Physik* 38 (1931), 173–198.

[3] J. B. Rosser, Extensions of some theorems of Gödel and Church, *Journal of Symbolic Logic* 1 (1936), 87–91.

[4] A. Tarski, Über einige fundamentale Begriffe der Metamathematik, *Comptes Rendus des Séances de la Société des Sciences et des Lettres de Varsovie, Classe III* 23 (1930), 22–29. (Consequence operators.)

[5] A. Tarski, Der Wahrheitsbegriff in den formalisierten Sprachen, *Studia Philosophica* 1 (1936), 261–405. (Satisfaction.)

[6] J. H. Lambert, Mémoire sur quelques propriétés remarquables des quantités transcendantes circulaires et logarithmiques, *Histoire de l'Académie Royale des Sciences et des Belles-Lettres de Berlin* (1768), 265–322.

[7] H. Jeffreys, *Scientific Inference*, 3rd ed., Cambridge University Press, 1973. (Cartwright's proof of the irrationality of π.)

[8] F. Lindemann, Über die Zahl π, *Mathematische Annalen* 20 (1882), 213–225.

[9] S. Wolfram, *A New Kind of Science*, Wolfram Media, 2002. (Rule 30.)

[10] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction – CADE 28*, LNCS 12699, Springer, 2021, 625–635.

[11] The mathlib Community, The Lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, 367–381.

[12] Lu Semita, Λ *at the Point of Adjudication: Conveyance, carried conditions, persistence, and the Navier–Stokes adjudication instrument*, successor laboratory edition R4, EmergenceByDesign, 14 September 2026; with the instrument file `Lambda_Adjudication_Instrument_R4.html` (Lean sources, checks, `#print axioms` outputs and browser receipts); the proof packet embedded there is `Lambda_Equal_Standard_R4_Proof_Packet.zip`.

[13] Lu Semita, *Deflation of Pharmakon-Based Delusion*, EmergenceByDesign, 9 September 2026 (Lean source, Lean Web run and verification receipt in the document).

[14] Lu Semita, *Lambda, Reference, and the Spatiotemporal Descriptive Frame*, receipt edition, EmergenceByDesign, 10 September 2026 (kernel `LambdaSpatiotemporalFrame.lean` in the document).

[15] Lu Semita, *Lambda / Reference / Descriptive Frames, Part II: Enumerative Resonance, Energy as Residue, and Physical Calibration*, EmergenceByDesign, 11 September 2026.

[16] Lu Semita, *Lambda, Part III*, kernel `LambdaPartIII.lean` (namespace `LambdaIII`), September 2026, as included in [12].

[17] Lu Semita, *Expression of the Dimensional Ladder as Dual-Framed Description According to the Lambda Principle of Irreducibility*, V2 with integrated verification receipt, EmergenceByDesign, 9 September 2026.

[18] *Closure at the Adjudication Seam*, R8.1, 24 October 2026; conceptual programme Lu Semita / EmergenceByDesign / MetaLabe, mathematical exposition and formal continuation devel-

oped with ChatGPT, prior parallax development Claude with Lu Semita; with the diagonal bridge R6 (23 October 2026) and the Rule 30 packets R1–R7.

[19] Lu Semita, *Class-Level Closure and Frame-Dual Non-Progress*, lab notes, version 1.0, EmergenceByDesign, 7 September 2026.

[20] Lu Semita and Orion, *The Double-Edged Receipt Standard: Quantifier Asymmetry, Scope, and Symmetric Burdens of Proof*, review draft v0.3, 23 September 2026.

[21] Lu Semita, *Equal Standard: Quantifier Structure, Scope, and Priority in Two Public Finite-Time Blowup Claims*, EmergenceByDesign, 12 September 2026; its two adjudication results are carried in [16].

[22] Lu Semita, *The Unwitnessable Verdict*, V2.1, EmergenceByDesign, August 2026, with its Lean demonstrator; *Before the Proof*, certified edition V1, 21 August 2026, with `BeforeTheProof_BU_FINAL.lean`.

[23] EmergenceByDesign, *Astra 15A: dimensional enlargement*, browser instrument and supplement, and *Astra Companion I: Coherence and Global Orientation Transport*, specification, September 2026.